LAPORAN PRAKTIKUM

POSTTEST 2

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Muhamad Radja Nur Akbar (2509106012)

Kelas (A1'25)

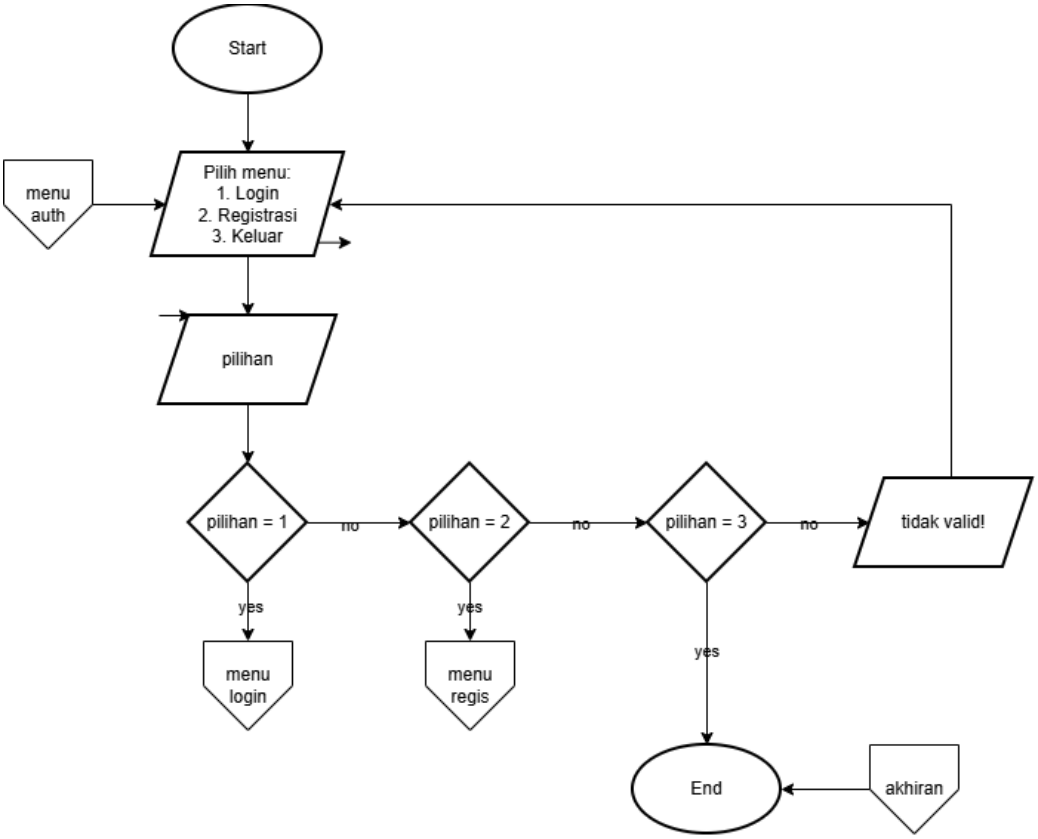
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

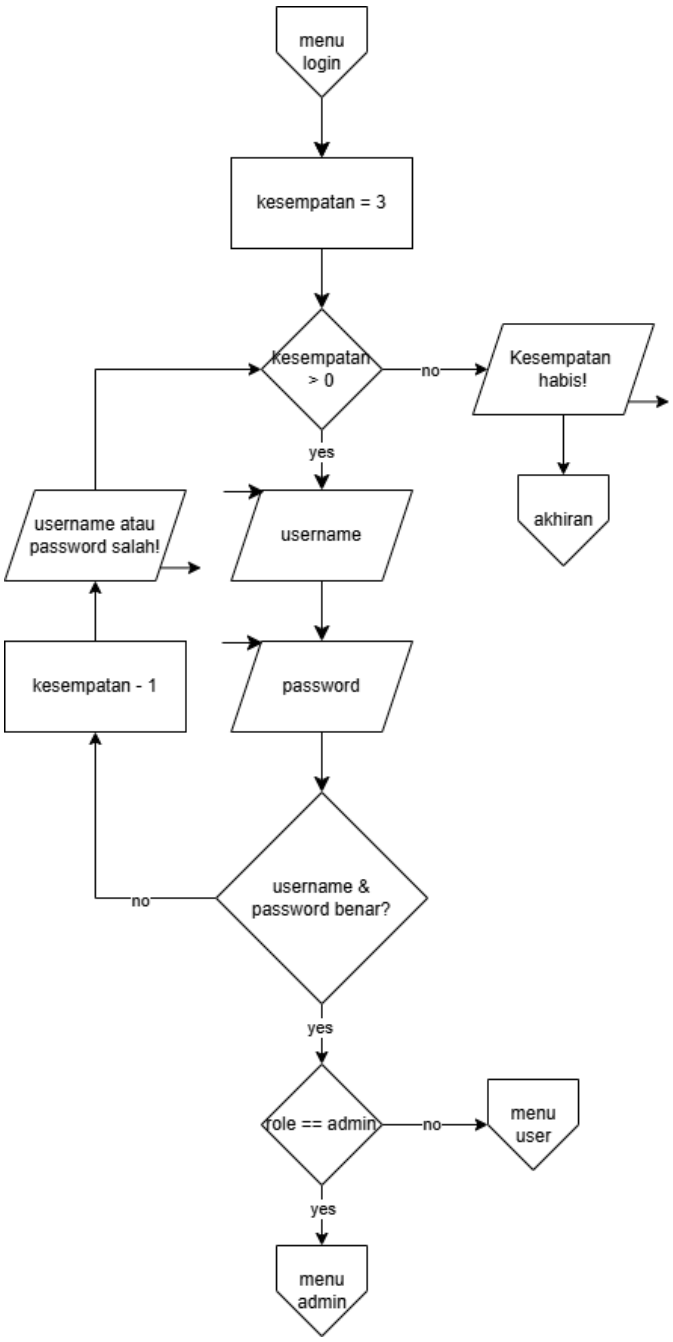
2026

1. Flowchart



Gambar 1.1 Menu Autentikasi

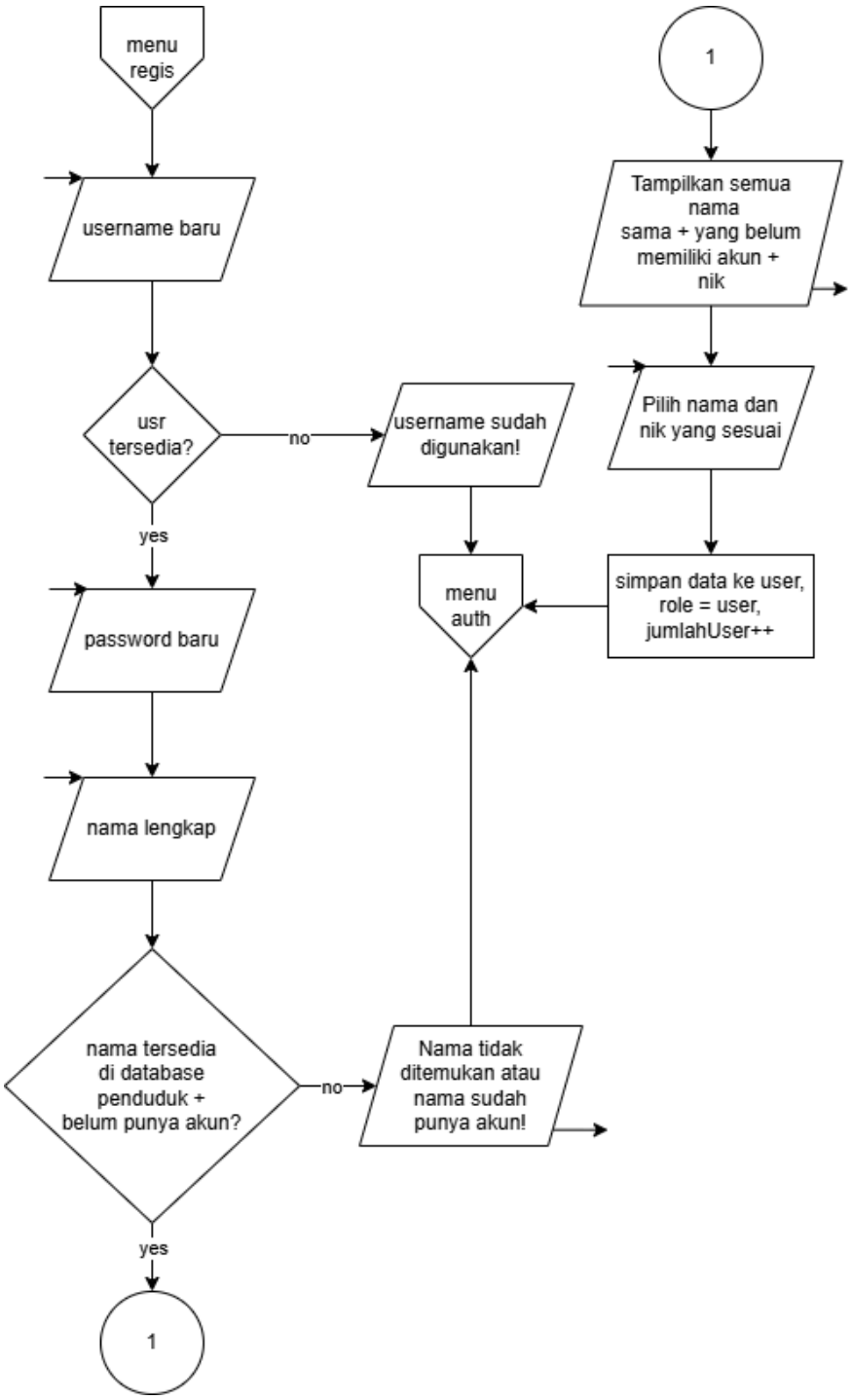
Terdapat tiga pilihan pada menu pertama ini, *login* untuk lanjut ke menu utama. Registrasi untuk membuat akun baru. Pilihan ketiga untuk mengakhiri program.



Gambar 1.2 Tampilan Login

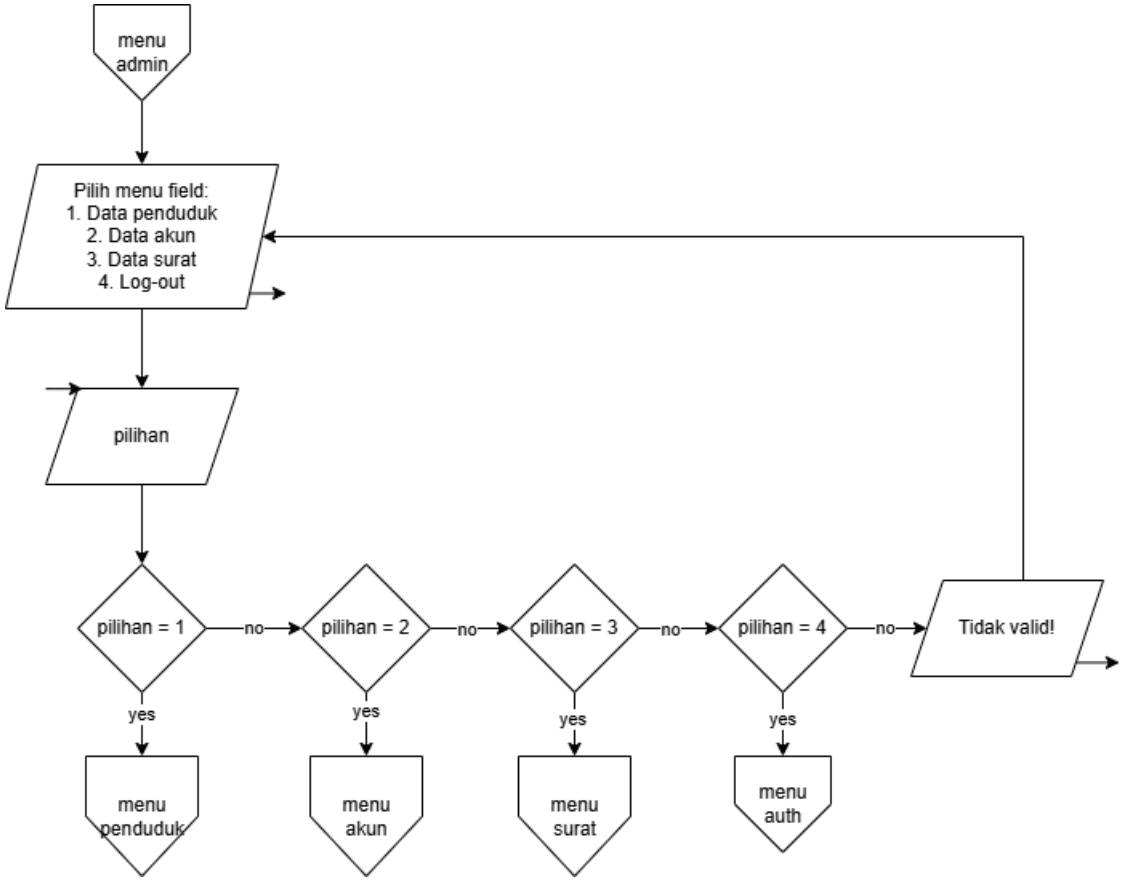
Pengguna punya 3 kesempatan untuk *login*, setiap kesalahan akan mengurangi kesempatan sebanyak 1. Jika habis program berhenti.

Ketika *login* sudah tepat, *role* dari akun tersebut dicek untuk membawa pengguna ke menu yang sesuai. Ada dua, menu admin dan menu *user*.



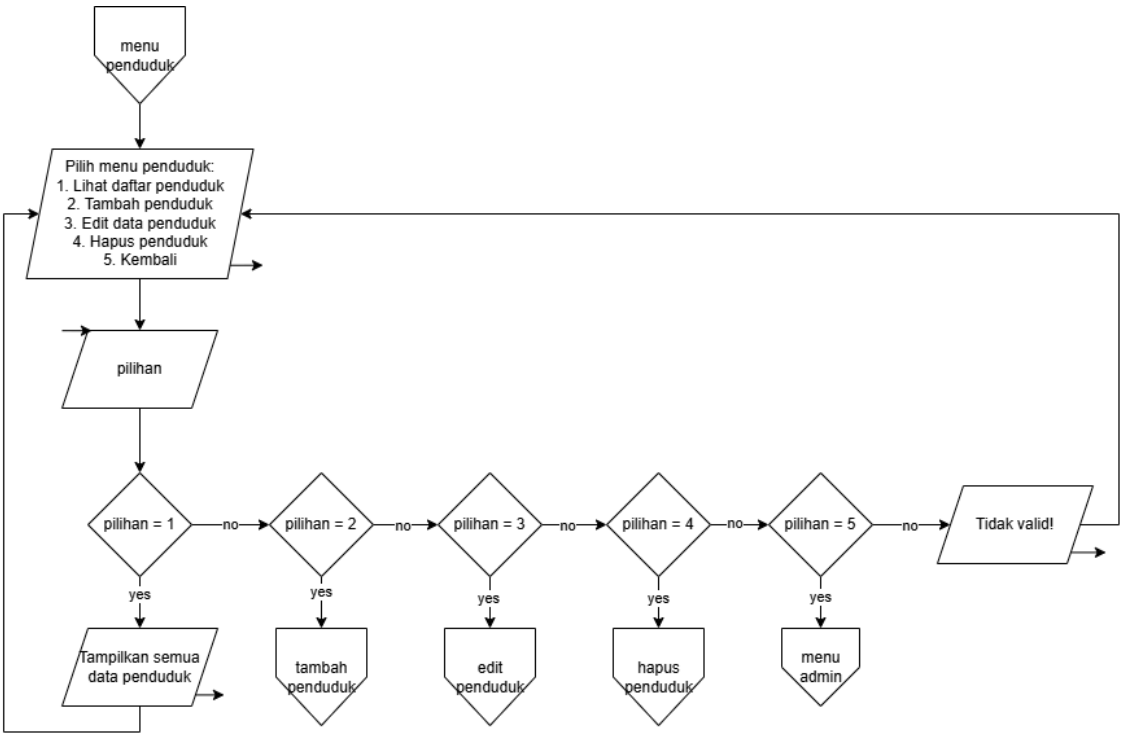
Gambar 1.3 Tampilan Registrasi

Di registrasi, masukkan *username* baru yang akan ditolak jika *username* sudah digunakan. Masukkan *password* dan nama lengkap. **Catatan:** pengguna hanya bisa registrasi jika mereka terdaftar di *database* penduduk. Jika terdapat nama yang sama dari *database* maka semua nama tersebut ditunjukkan beserta NIK supaya pengguna bisa memilih data yang tepat. Terakhir simpan *username* dan *password* dengan *role* bawaan adalah *user.*



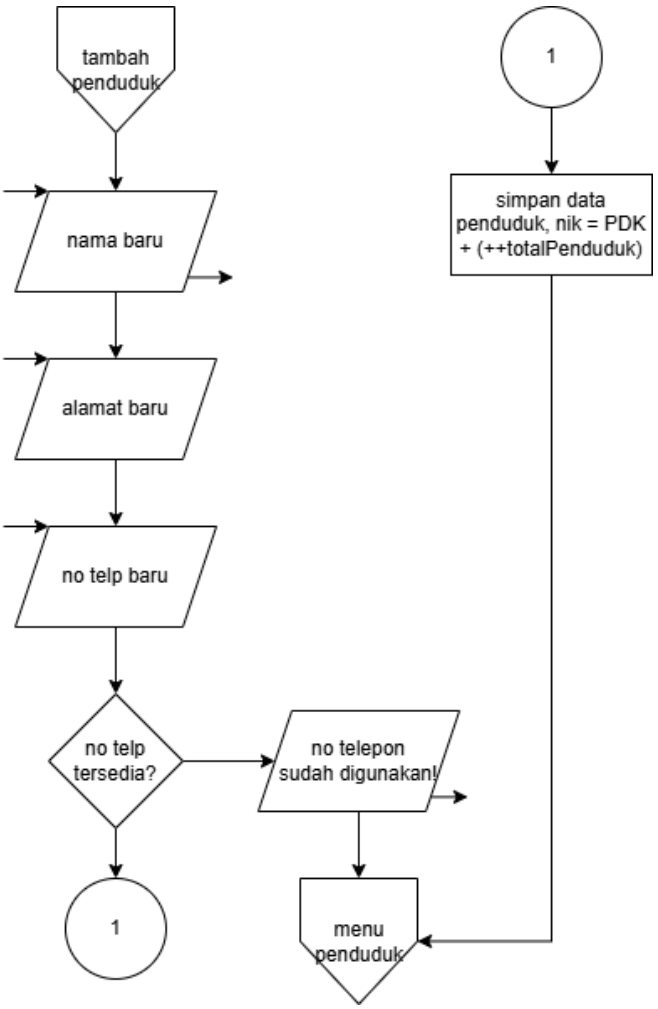
Gambar 1.4 Menu Utama Admin

Admin memiliki 3 wewenang dalam kepengurusan balai kota, yaitu mengurus data penduduk, data akun, dan data surat. Admin hanya perlu memilih salah satu menu untuk melanjutkan.



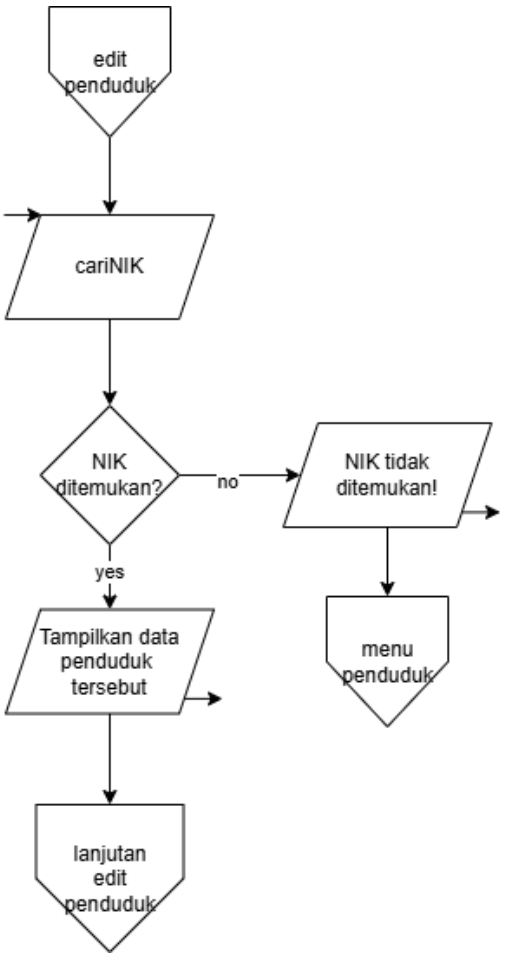
Gambar 1.5 Tampilan Menu Data Penduduk

Dalam menu ini admin punya 4 pilihan dalam mengatur data penduduk. Melihat daftar penduduk, menambah penduduk, mengedit data penduduk, dan menghapus penduduk.



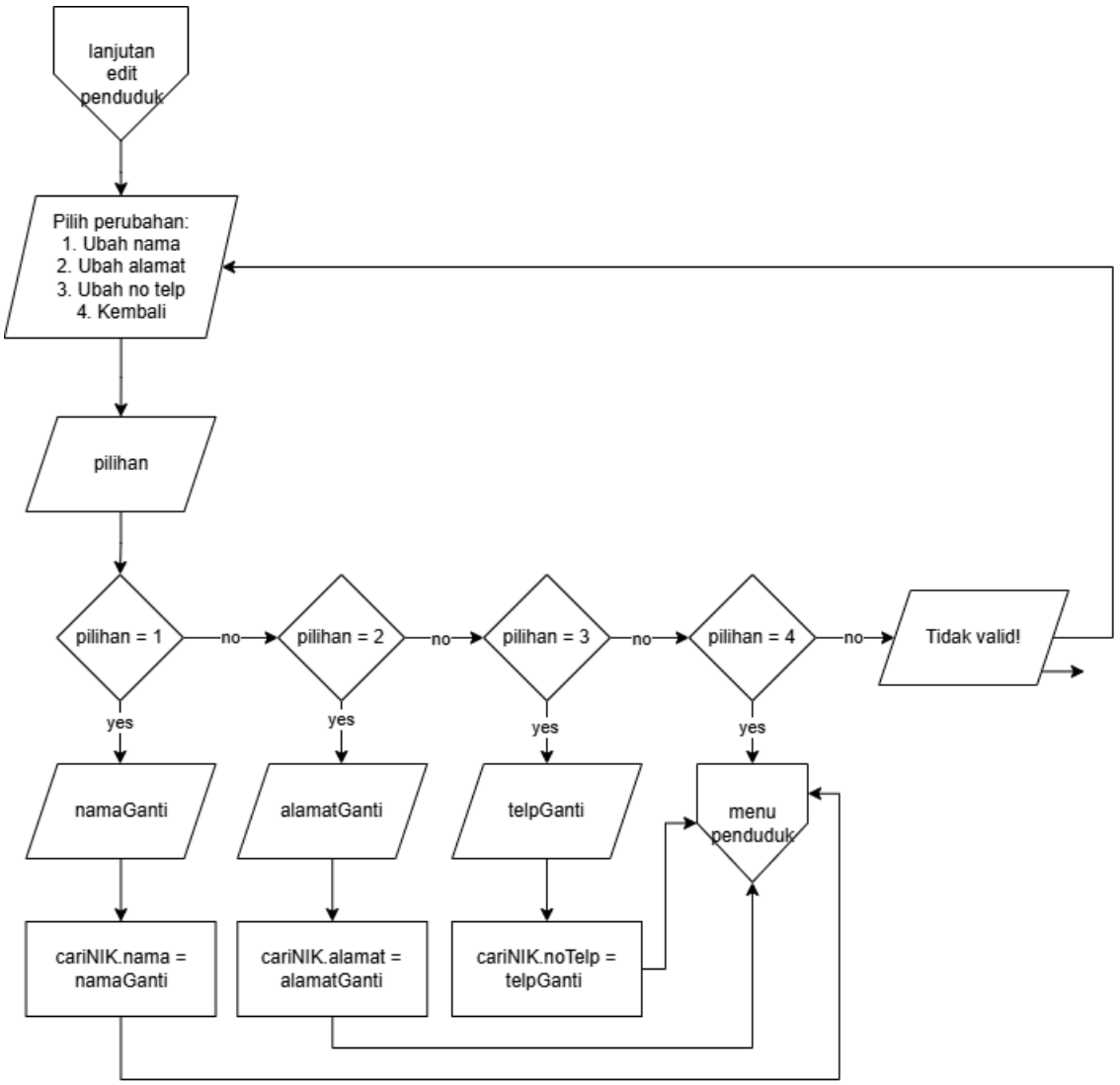
Gambar 1.6 Menambah Penduduk

Penduduk memiliki 4 *subfield* yaitu NIK sebagai variabel identitas utama, nama, alamat, dan nomor telepon. Dalam menu di atas nomor telepon juga merupakan variabel unik yang tidak bisa sama satu sama lain. Ketika semua *subfield* sudah diisi variabel NIK akan otomatis terisi dengan menggunakan 3 huruf "PDK" ditambah total penduduk saat ini.



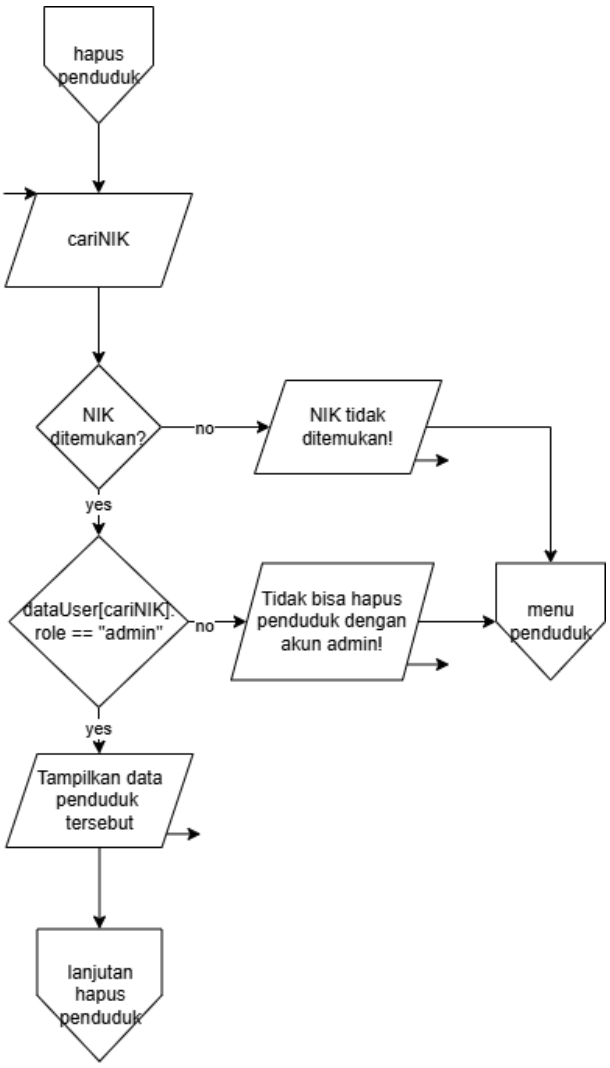
Gambar 1.7 Mengedit Data Penduduk

Sebelum mengedit data penduduk, admin harus memasukkan NIK dari penduduk yang datanya akan diubah. Jika NIK tidak ditemukan admin akan kembali ke menu sebelumnya.



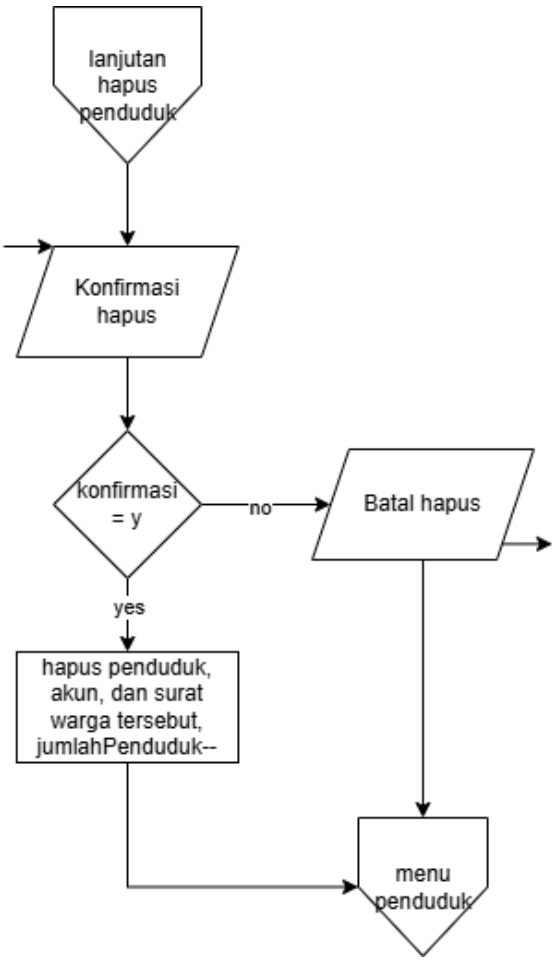
Gambar 1.8 Mengubah Subfield

Admin dapat mengubah 3 *subfield* data penduduk. Prosesnya sama ketika menambah penduduk termasuk nomor telepon unik. Setelah mengganti data lama akan ditimpa dengan data baru.



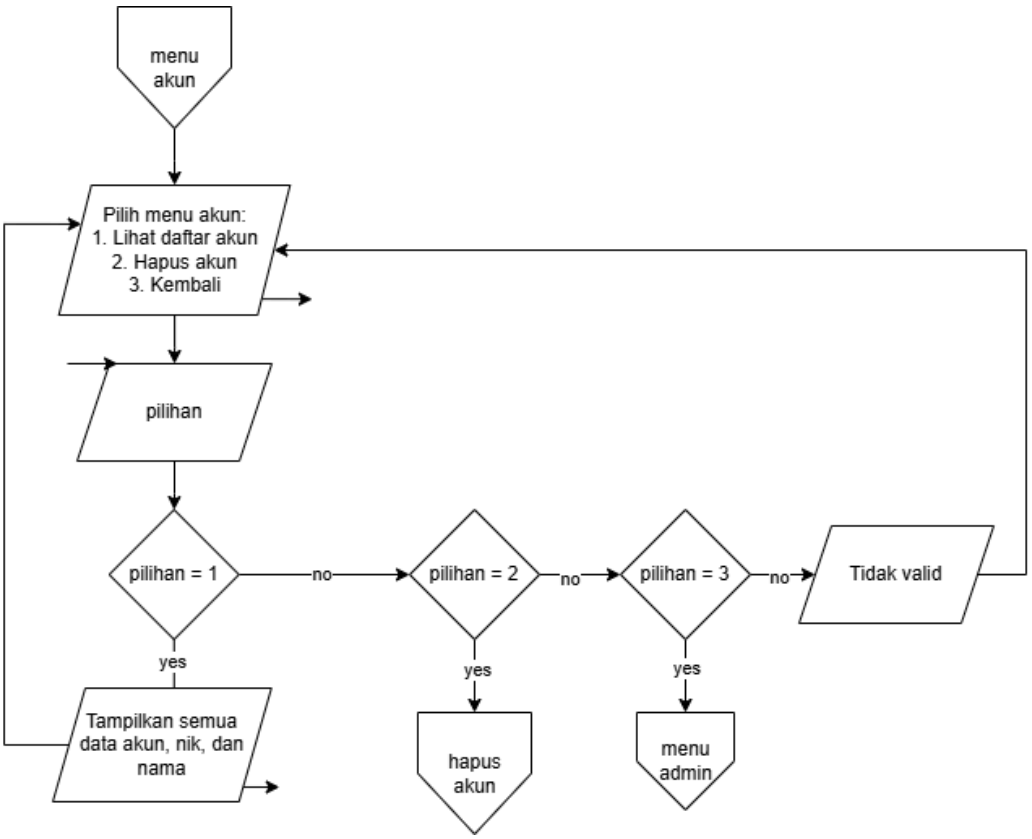
Gambar 1.9 Menghapus Data Penduduk

Seperti saat akan mengedit data penduduk, admin juga harus memasukkan NIK yang diinginkan untuk memulai proses penghapusan. Adapun syarat tambahan yaitu admin tidak dapat menghapus akun sesama admin.



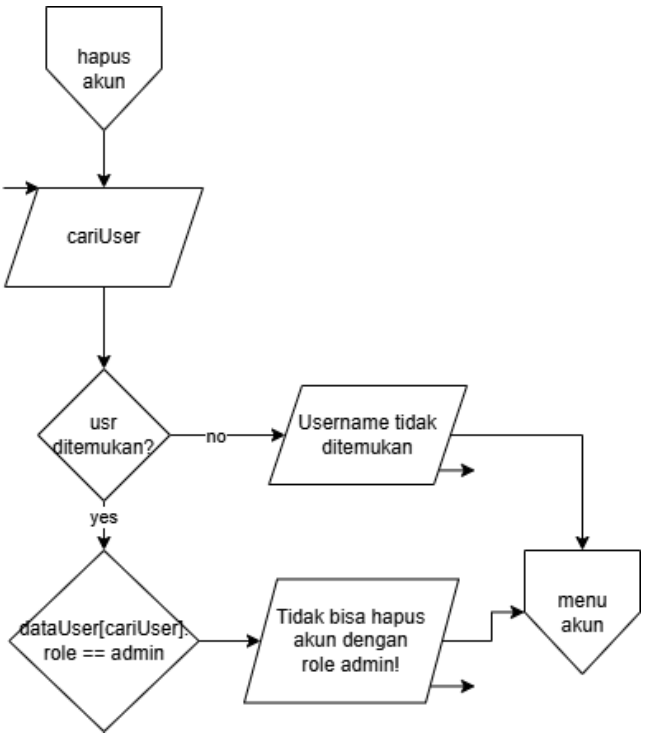
Gambar 1.10 Hapus Berantai

Admin diminta konfirmasi sebelum betul-betul menghapus data penduduk. Ketika data suatu penduduk dihapus maka akun dan semua surat dari penduduk tersebut juga akan dihapus.

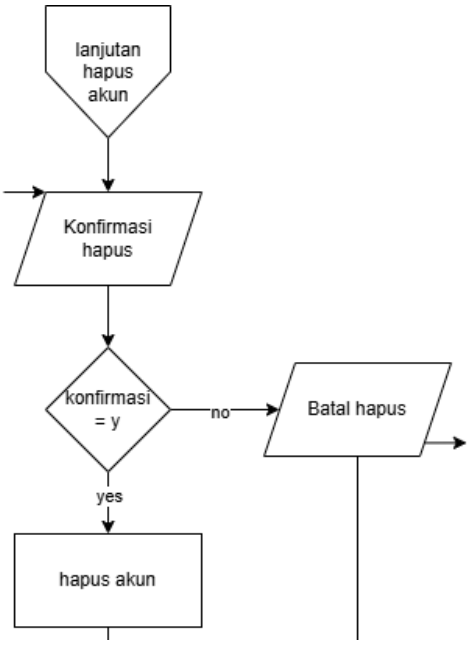


Gambar 1.11 Menu Field Akun

Setelah selesai di menu *field* penduduk, admin juga bisa memanipulasi akun-akun yang terdaftar. Memilih pilihan 1 akan menampilkan semua akun yang terdaftar beserta *role*, NIK, dan nama.

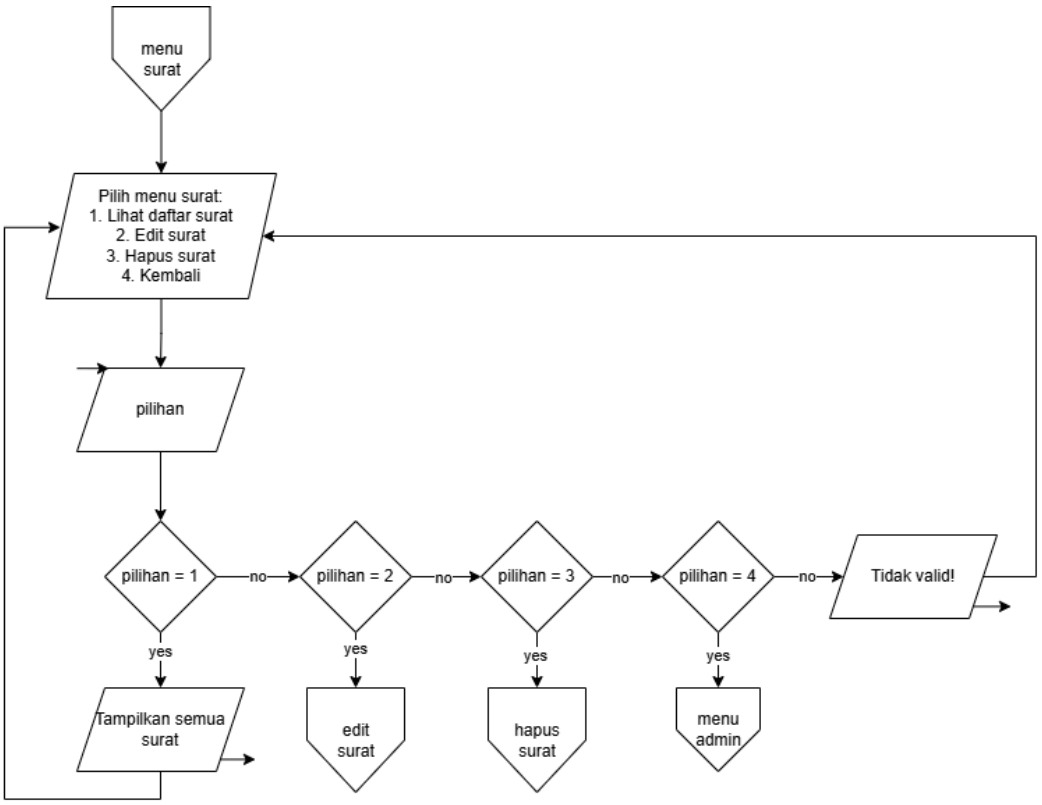


Gambar 1.12 Hapus Akun



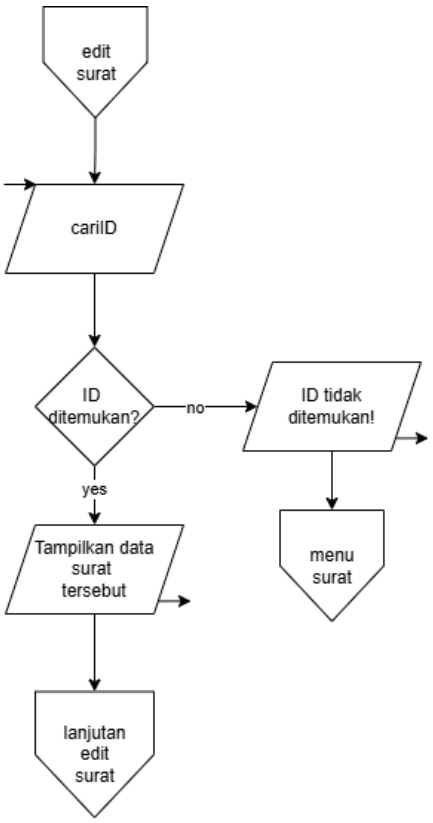
Gambar 1.12 Konfirmasi Hapus

Sama halnya saat menghapus penduduk, admin harus memasukkan NIK yang diinginkan, konfirmasi, dan akun tersebut akan dihapus.



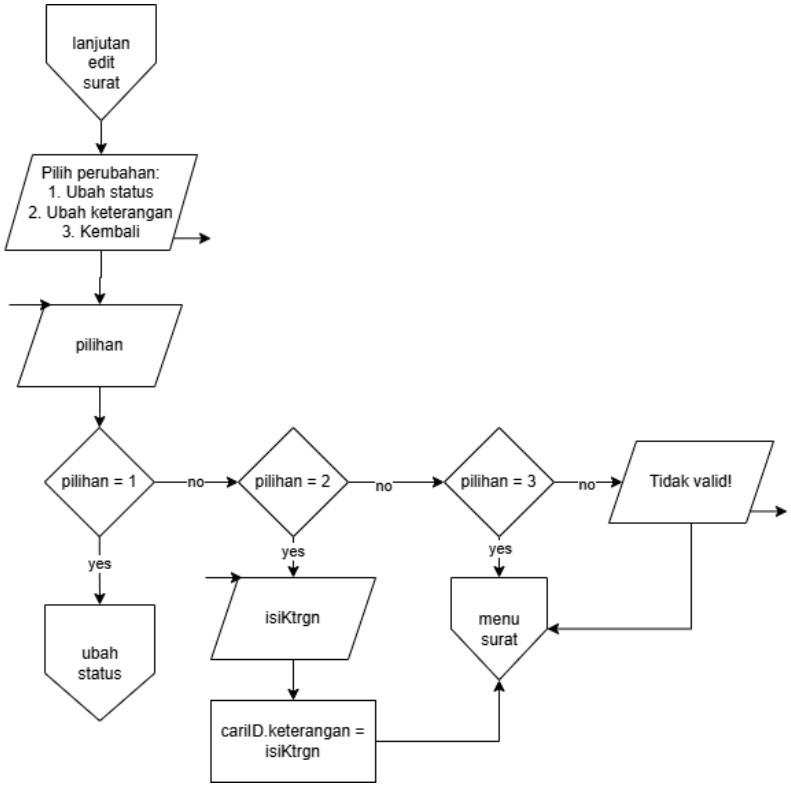
Gambar 1.13 Menu Field Surat

Pada pilihan ketiga admin dapat mengubah data pada surat-surat ajuan. Data surat punya 6 subfield, yaitu ID surat, NIK pemohon, jenis surat, keperluan, status, keterangan. Admin dapat melihat daftar surat, mengedit surat, dan menghapus surat.



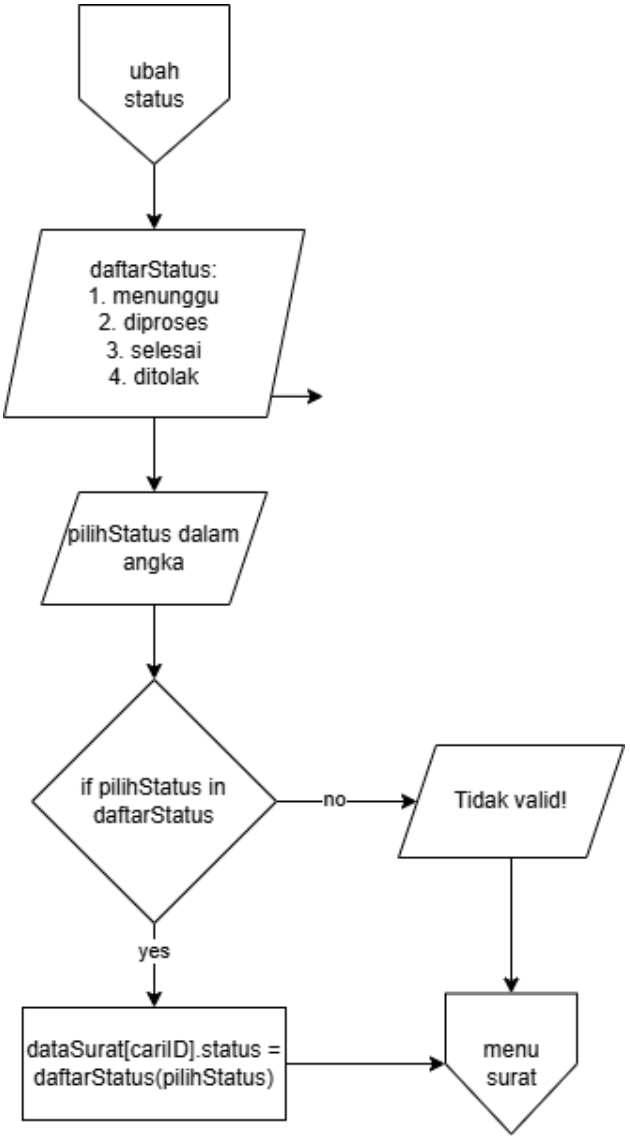
Gambar 1.14 Edit Surat

Admin harus memasukkan ID surat yang akan diubah datanya, jika ID ditemukan admin bisa mengubah data surat tersebut.



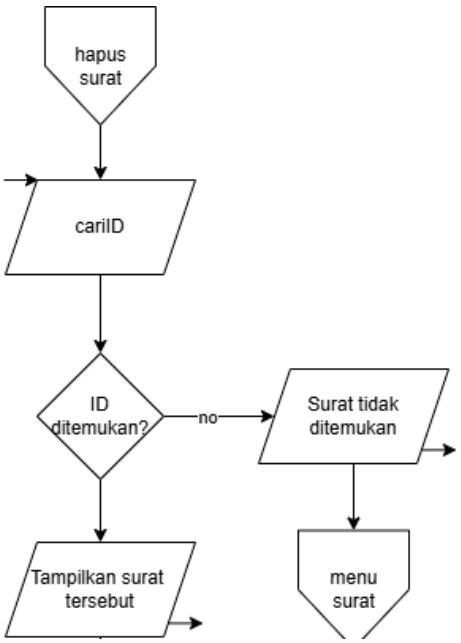
Gambar 1.15 Menu Perubahan Data Surat

Admin tidak dapat mengubah data surat sepenuhnya, hanya di bagian status dan keterangan. Status menunjukkan perubahan surat pada waktu ke waktu, sementara keterangan menunjukkan kepada pemohon apa yang terjadi sebenarnya dengan surat mereka.

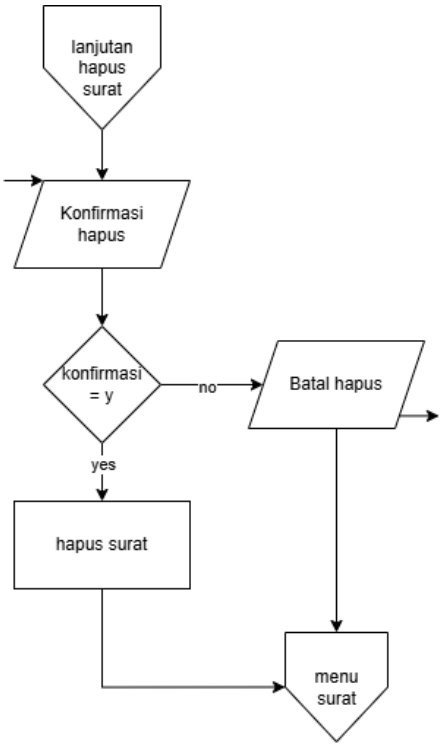


Gambar 1.16 Ubah Status

Status sendiri sudah memiliki ketetapan, jadi admin hanya bisa memilih satu dari 4 status yang ada yang kemudian menimpa status saat ini dengan yang baru.

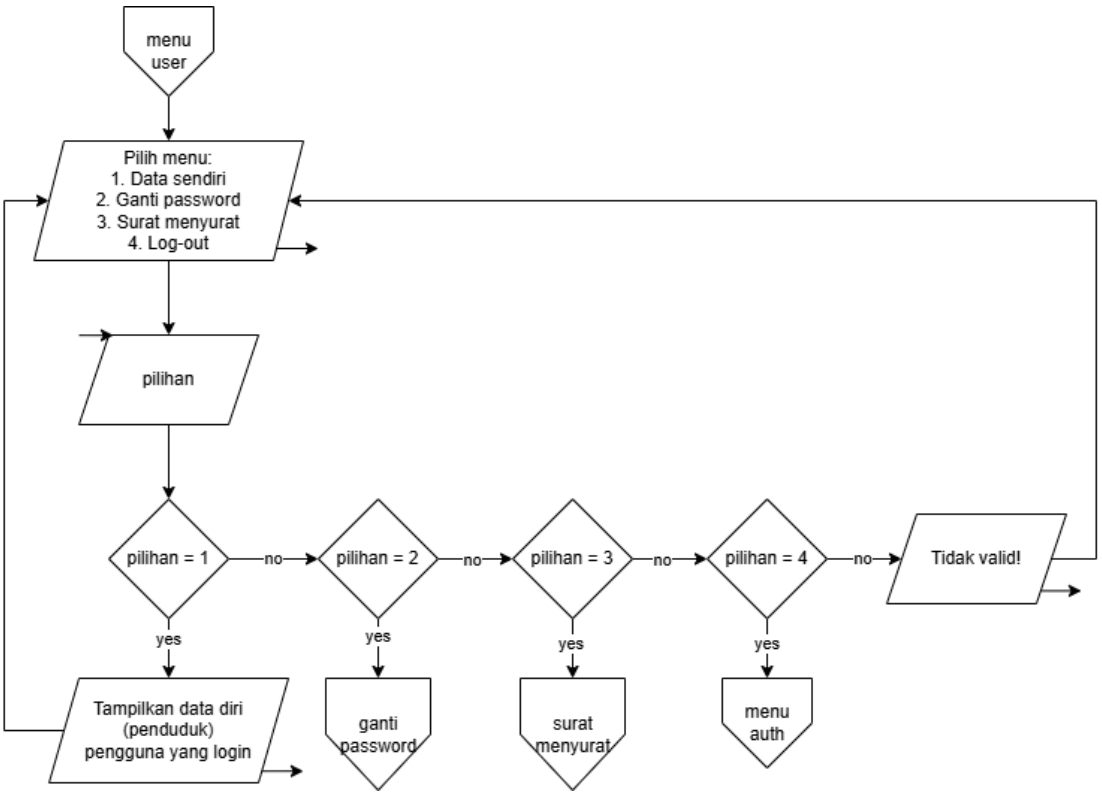


Gambar 1.17 Hapus Surat



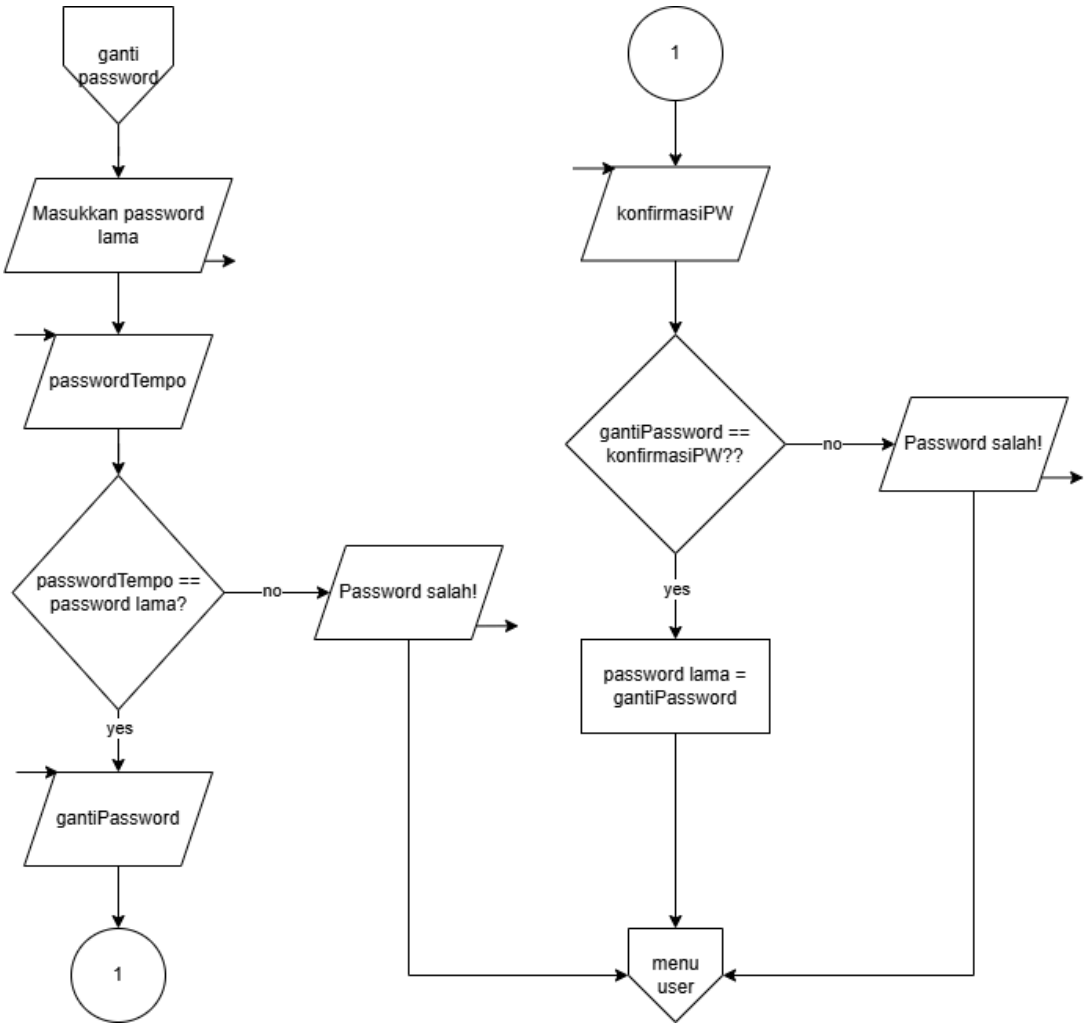
Gambar 1.18 Konfirmasi Hapus Surat

Sama seperti sebelum-sebelumnya, masukkan ID surat terlebih dahulu, meminta konfirmasi, lalu surat dihapus sepenuhnya.



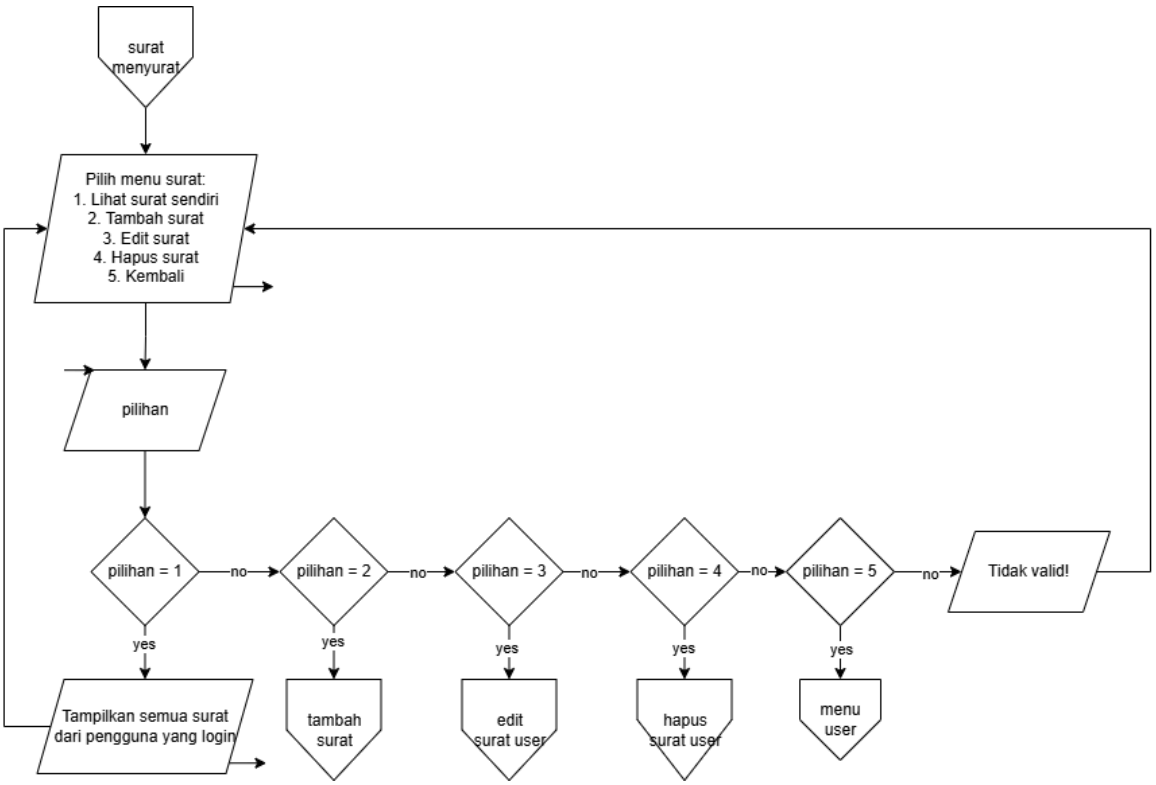
Gambar 1.19 Tampilan Menu User

Selain admin, ada juga *role user*. *Role* ini hanya bisa memanipulasi data yang terikat dengan dirinya sendiri dan memiliki beberapa batasan. User punya 3 menu utama, menampilkan data mereka, mengganti *password*, dan yang paling utama menu surat menyurat.



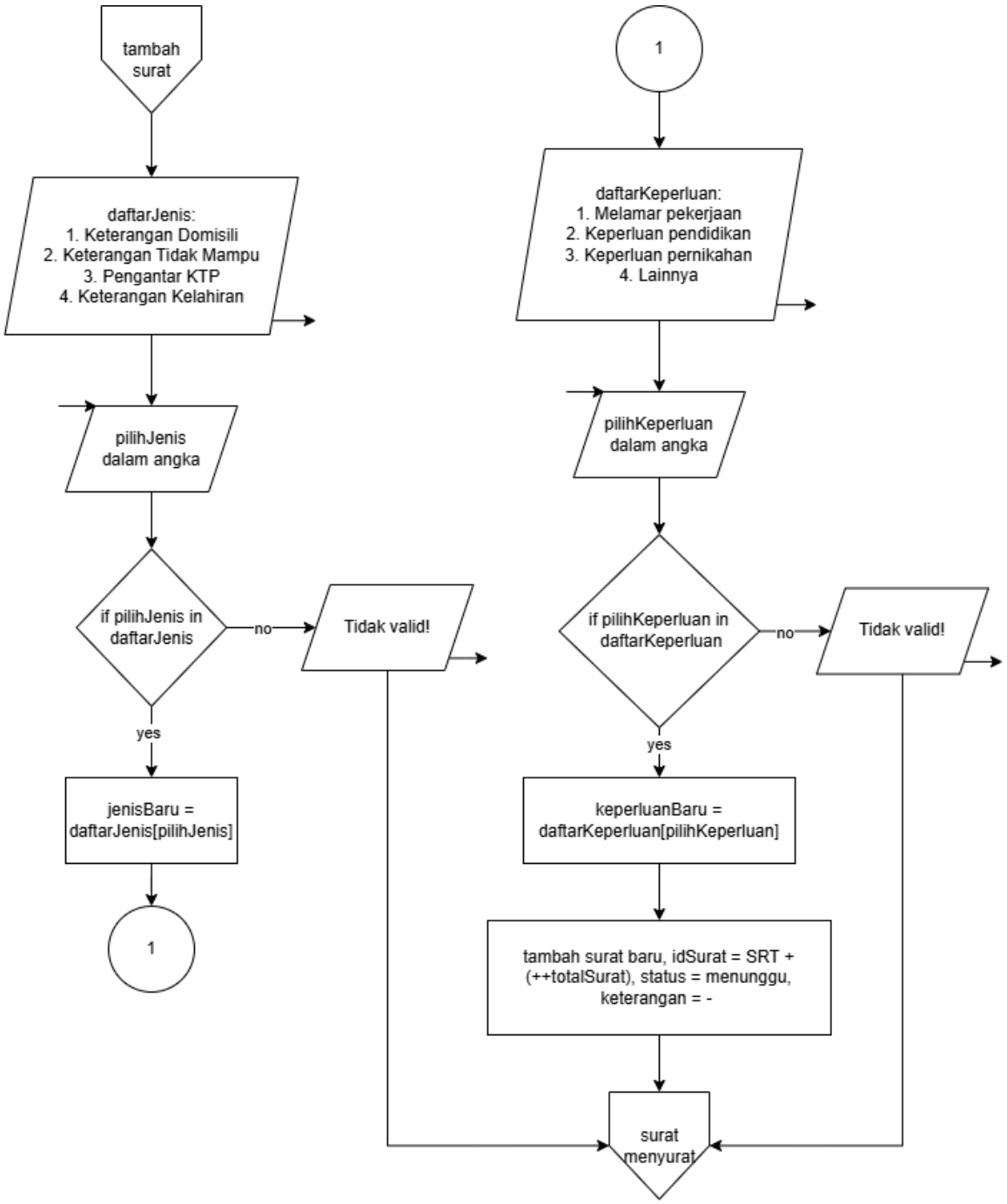
Gambar 1.20 Mengganti Password

Untuk menjaga keamanan ketika mengganti *password*, *user* harus memasukkan *password* mereka saat ini. Setelah verifikasi selesai barulah *user* bisa memasukkan *password* baru, adapun *user* juga harus mengkonfirmasi *password* baru mereka dengan memasukkan password yang sama sekali lagi. Jika semua langkah di atas berjalan dengan tepat dan benar, barulah *password* lama akan ditimpa dengan yang baru.



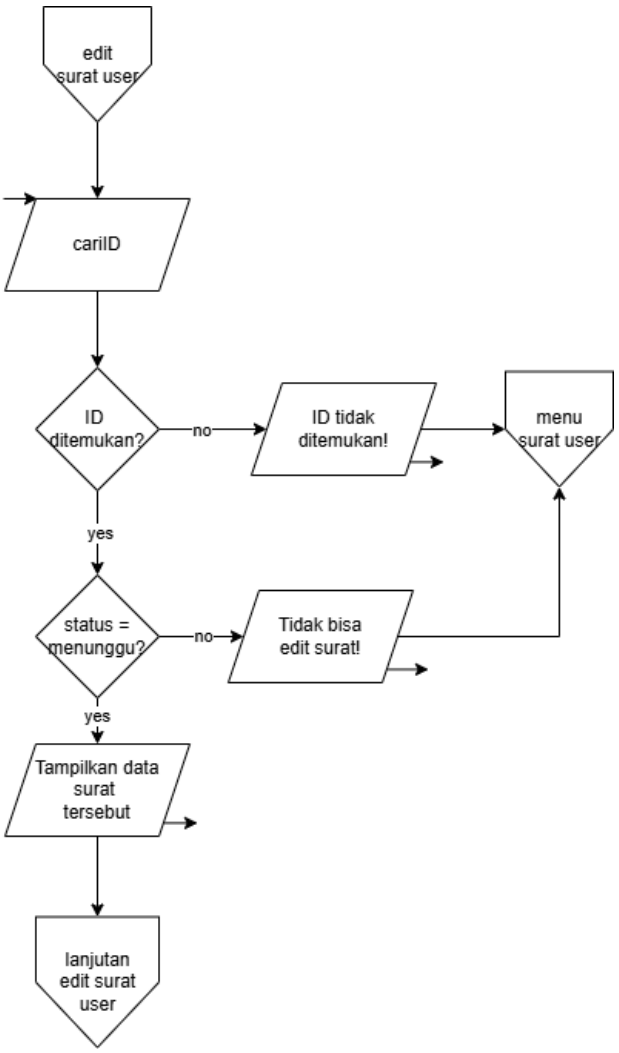
Gambar 1.21 Menu Surat Menyurat

Disini menu utama *user*, mereka secara bebas dapat membuat surat, melihat semua surat yang pernah mereka buat/mohon (pilihan 1), mengedit dan menghapus surat mereka.



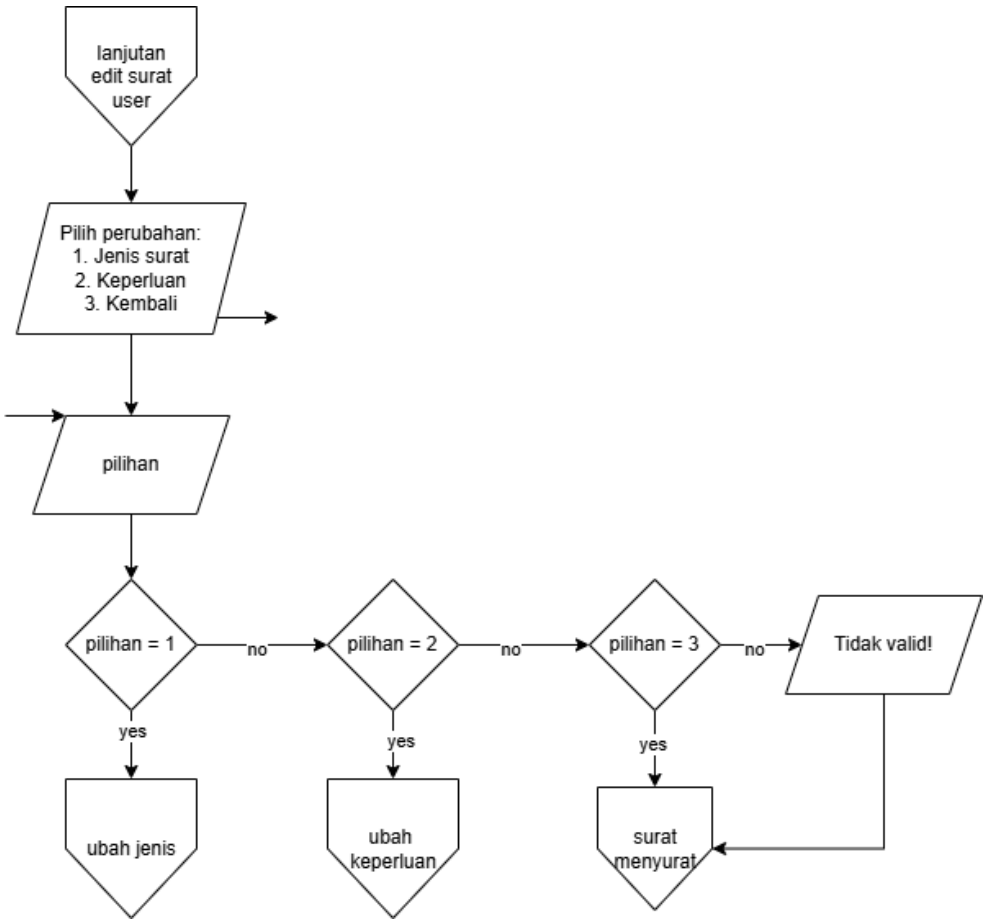
Gambar 1.22 Membuat Surat

Dalam membuat sura, *user* hanya bisa memilih jenis dan keperluan surat sesuai daftar. Ketika semua sudah selesai, barulah program menambah surat tersebut. ID surat otomatis bertambah seperti NIK, status akan ditetapkan ke menunggu dan isi keterangan kosong yang nantinya akan diubah admin.



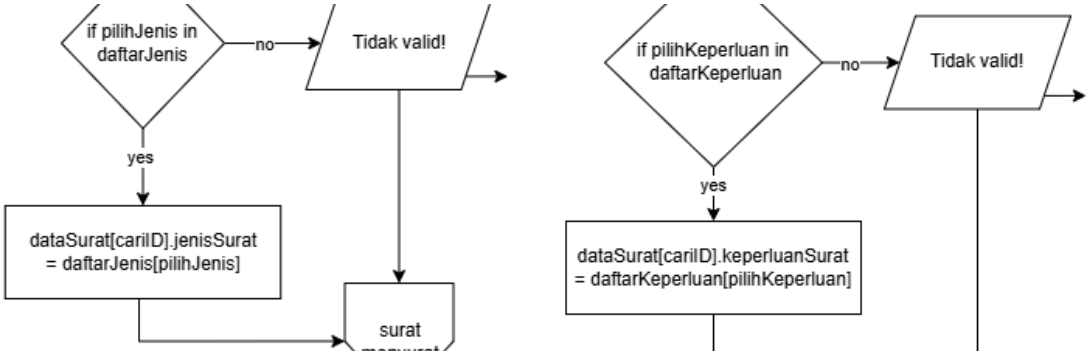
Gambar 1.23 Mengedit Surat Versi User

Sama halnya ketika admin ingin mengedit surat, *user* juga harus memasukkan ID surat yang ingin diubah. Namun disini ada persyaratan, *user* hanya dapat mengedit surat selama status surat tersebut masih menunggu, selebihnya akan ditolak program.



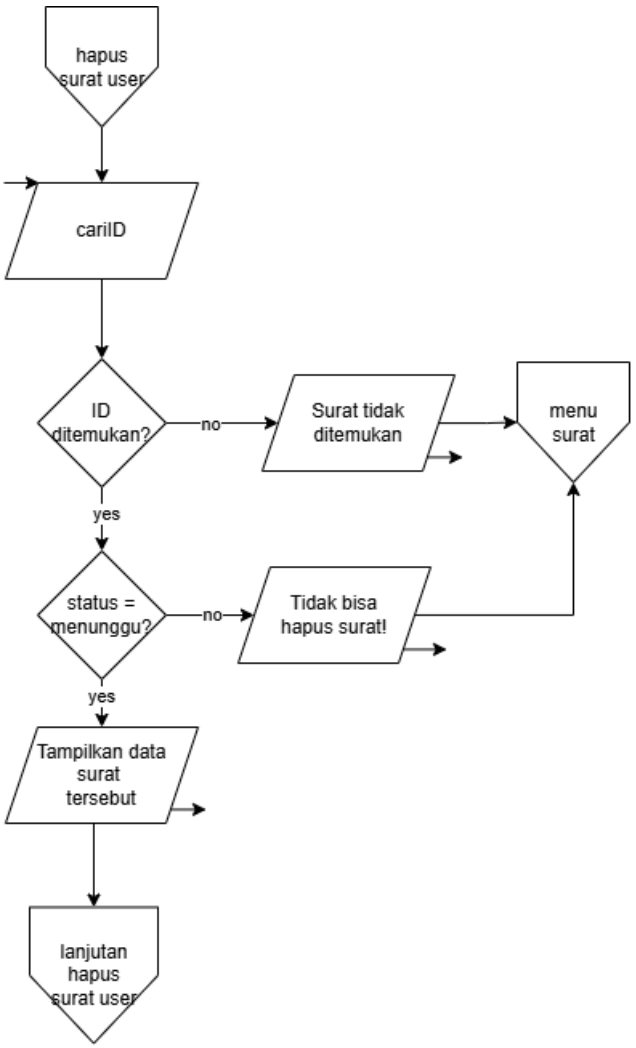
Gambar 1.24 Menu Perubahan Surat User

User hanya bisa mengedit jenis dan keperluan surat, dengan cara yang hampir sama ketika admin mengubah status surat.



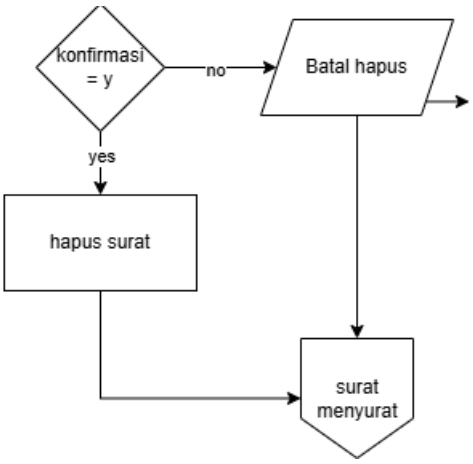
Gambar 1.25 Ubah Jenis dan Keperluan

Jenis/keperluan baru akan menimpa yang lama dan *user* kembali ke menu surat menyurat.



Gambar 1.26 Hapus Surat Versi User

Proses sama saat mengedit surat, surat hanya bisa dihapus jika status masih menunggu.



Gambar 1.27 Konfirmasi Hapus Surat

Setelah *user* mengkonfirmasi penghapusan barulah surat tersebut dihapus sepenuhnya.

2. Deskripsi Singkat Program

Program administrasi balai kota dikelola oleh sekumpulan admin yang memegang database penduduk-penduduknya. Mereka dapat memanipulasi database tersebut. Selain itu mereka juga mengatur akun-akun non-admin. Tujuan utama program ini yaitu kegiatan surat menyurat, user atau pengguna biasa dibebaskan dalam permohonan surat yang ditujukan ke administrasi balai kota. Admin tidak bisa mengubah keaslian surat-surat tersebut melainkan hanya membantu si pemohon dengan terus memperbarui status dan keterangan yang ada pada surat.

3. Source Code

Catatan: *Source Code* yang dijelaskan hanya berisi *Source Code* penting, kode lebih detail ada di *file* cpp.

```
struct Penduduk {
    string nik;
    string nama;
    string alamat;
    string noTelp;
};
struct User {
    string usr;
    string pw;
    string role;
    Penduduk dataDiri;
};
struct Surat {
    string idSurat;
    string nikPemohon;
    string jenisSurat;
    string keperluan;
    string status;
    string keterangan;
};
```

Source Code 3.1 Kumpulan Struct

Untuk mempermudah program, dibuat 3 *struct* utama. Pada *struct* 'User' menggunakan *nested struct* 'Penduduk' dengan nama variabel 'dataDiri'. Ketiga *struct* ini akan digunakan sebagai tipe data pada *array of struct* yang menampung seluruh data, dan digunakan pada proses autentikasi dan CRUD.

```
Penduduk dataPenduduk[100];
    User dataUser[100];
    Surat dataSurat[100];
    int jumlahPenduduk = 2;
    int jumlahUser = 2;
    int jumlahSurat = 2;
    int totalSurat = 2;
    int totalPenduduk = 2;
    int indexUser = -1;
    int pilihan;
```

Source Code 3.2 Inisialisasi Array of Struct dan Variabel

Kita menggunakan 3 *array of struct* untuk menyimpan data. Kemudian ada juga beberapa variabel seperti jumlah menunjukkan kuantitas saat ini. Lalu variabel total menunjukkan kuantitas secara keseluruhan, variabel ini tidak akan bisa berkurang. Adapun flag 'indexUser' ditetapkan di -1 menunjukkan saat ini sedang tidak menunjuk indeks manapun. Terakhir 'pilihan' sebagai variabel yang menampung pilihan dari semua pemilihan menu.

```
dataPenduduk[0] = {"PDK1", "radja", "gg bersatu", "08111"};
    dataPenduduk[1] = {"PDK2", "amat", "gg bersama", "08112"};

    dataUser[0] = {"r", "012", "admin", dataPenduduk[0]};
    dataUser[1] = {"a", "011", "user", dataPenduduk[1]};

    dataSurat[0] = {"SRT1", "PDK2", "Surat Keterangan Tidak Mampu",
"Keperluan Pendidikan", "diproses",
                    "Dalam pengecekkan"};
    dataSurat[1] = {"SRT2", "PDK2", "Surat Keterangan Tidak Mampu",
"Keperluan usaha", "menunggu", "-"};
```

Source Code 3.3 Data Dummy

Untuk mengecek program disini ada beberapa data dummy dari masing-masing *array*.

```
cout << "1. Login\n";
        cout << "2. Register\n";
        cout << "3. Keluar\n";
```

Source Code 3.4 Menu Autentikasi

```
if (!(cin >> pilihan)) {
            cin.clear();
```

```
            cin.ignore(100, '\n');
            cout << "Input tidak valid!\n\n";
            continue;
```

Source Code 3.5 Blok Kode saat Memilih Menu

Karena program ini berisi permintaan menu yang sangat banyak, blok kode di atas akan selalu digunakan. Ketika pengguna memasukkan input selain angka maka program langsung meminta pengguna input ulang dan notifikasi kesalahan input. Ketika 'cin' gagal membaca ia akan beku dan 'cin.clear()' yang mengembalikannya ke semula. Adapun 'cin.ignore()' akan membuat sisa karakter pada buffer, maks 100 karakter atau saat membaca enter/newline. 'Continue' melewati sisa kode di *switch* dan kembali ke menu sebelumnya.

```
case 1: {
                int kesempatan = 3;
                string username, password;
                string roleUser = "";
                bool loginSukses = false;

                cin.ignore();
                while (kesempatan > 0 && !loginSukses) {
                    cout << "Silahkan login untuk melanjutkan\n";
                    cout << "Username: ";
                    getline(cin, username);
                    cout << "Password: ";
                    getline(cin, password);
                    bool ditemukan = false;

                    for (int i = 0; i < jumlahUser; i++) {
                        if (username == dataUser[i].usr && password ==
dataUser[i].pw ) {
                            loginSukses = true;
                            roleUser = dataUser[i].role;
                            indexUser = i;
                            ditemukan = true;
                            cout << "\nLogin berhasil! Selamat datang, " <<
username << "!\n\n";
                            break;
```

Source Code 3.6 Case Login

Pengguna punya 3 kesempatan untuk *login* yang jika habis program berhenti. Permintaan *username* dan *password* menggunakan *getline* untuk mengambil karakter spasi sebagai kebebasan pengguna dalam membuat akun.

Kemudian perulangan *for* dilakukan dengan mengakses ‘jumlahUser’, ketika akun yang tepat ditemukan, *flag* ‘loginSukses’ dan ‘ditemukan’ akan menjadi *true*. Jika kedua variabel ini masih bernilai *false* selama sesi login maka program tidak akan lanjut.

Role akan diambil dari *role* pengguna untuk melanjutkan ke menu *role* yang sesuai. Sementara ‘indexUser’ akan mengambil ‘i’ pada iterasi saat ini/indeks dari akun yang sesuai. Ini berfungsi ketika akan mengakses data sendiri, buat surat, dan *log-out*.

```
string usernameBaru, passwordBaru, namaCari;
                cout << "Masukkan username baru: ";
                cin.ignore();
                getline(cin, usernameBaru);
                if (usernameBaru == "") {
                    cout << "Tidak boleh kosong!\n";
                    break;
                }
                bool available = false;
                for (int i = 0; i < jumlahUser; i++) {
                    if (usernameBaru == dataUser[i].usr) {
                        available = true;
                        break;
```

Source Code 3.7 Case Register Username

Pada case kedua yaitu register pengguna harus memasukkan *username* dan *password* yang diinginkan. *Username* sendiri tidak boleh kosong dan akan ditolak jika *username* yang dimasukkan sudah digunakan (variabel ‘available’).

```
cout << "Masukkan password baru: ";
                getline(cin, passwordBaru);
                if (passwordBaru.length() < 3) {
                    cout << "Panjang password minimal 3 karakter!\n";
                    break;
```

Source Code 3.8 Input Password

Sementara *password* harus minimal berjumlah 3 karakter.

```
cout << "Masukkan nama lengkap: ";
                getline(cin, namaCari);

                int hasilCari[100];
                int jumlahHasil = 0;
                for (int i = 0; i < jumlahPenduduk; i++) {
                    if (dataPenduduk[i].nama == namaCari) {
```

```
                        bool punyaAkun = false;
                        for (int j = 0; j < jumlahUser; j++) {
                            if (dataUser[j].dataDiri.nik ==
dataPenduduk[i].nik) {
                                punyaAkun = true;
                                break;
                            }
                        }
                        if (!punyaAkun) {
                            hasilCari[jumlahHasil] = i;
                            jumlahHasil++;
```

Source Code 3.9 Bagian Penting Registrasi

Ini bagian penting dalam proses regis. Program ini didesain bahwa pengguna hanya bisa membuat akun jika data penduduk mereka terdaftar pada *database* penduduk. Jika mereka terdaftar mereka bisa memilih data yang sesuai.

Pada blok kode di atas setelah meminta *username* dan *password*, nama lengkap pengguna juga diminta untuk dicek pada *database*. Tujuannya nanti untuk mencegah kebingungan terhadap duplikasi nama yang sama.

Program menggunakan *array* sementara 'hasilCari' untuk menampung semua nama yang sama dan 'jumlahHasil' yaitu jumlah nama yang sama. Perulangan 'i' dilakukan pada data penduduk dengan membandingkan semua nama pada data tersebut dengan nama lengkap saat regis, termasuk duplikasi nama.

Perulangan 'j' dilakukan pada *database* user tepatnya *nested struct* 'dataDiri' untuk memperkecil duplikasi nama dengan menyeleksi penduduk yang sudah memiliki akun. Mereka yang gagal seleksi akan ditampung pada *array* 'hasilCari' sementara 'jumlahHasil' terus bertambah hingga perulangan 'j' dan 'i' berakhir.

```
if (jumlahHasil == 0) {
                    cout << "Nama tidak ditemukan atau nama sudah punya
akun!";
                    break;
                }
                int indexPenduduk = -1;
                if (jumlahHasil == 1) {
                    indexPenduduk = hasilCari[0];
                } else {
                    cout << "Ditemukan " << jumlahHasil << " penduduk dengan
nama sama: \n";
                    for (int i = 0; i < jumlahHasil; i++) {
                        cout << i + 1 << ". " <<
```

```
dataPenduduk[hasilCari[i]].nama <<
                        " - " << dataPenduduk[hasilCari[i]].nik << "\n";
                    }
                    int pilihPenduduk;
                    cout << "Pilih nomor: ";
                    cin >> pilihPenduduk;
                    if (pilihPenduduk < 1 || pilihPenduduk > jumlahHasil) {
                        cout << "Pilihan tidak valid!\n";
                        break;
                    }
                    indexPenduduk = hasilCari[pilihPenduduk-1];
                }

                dataUser[jumlahUser] = {usernameBaru, passwordBaru, "user",
dataPenduduk[indexPenduduk]};
                jumlahUser++;
                cout << "Registrasi berhasil!\n";
                cout << "Nik kamu: " << dataPenduduk[indexPenduduk].nik <<
"\n";
                break;
```

Source Code 3.10 Pemilihan Nama dan NIK yang Tepat

Langkah berikutnya pemilihan nama yang tepat, kita menggunakan 'indexPenduduk' = -1 sebagai *flag* awal. Jika sebelumnya 'jumlahHasil' kosong/0, maka nama dinyatakan tidak (belum terdaftar) atau nama tersebut sudah memiliki akun.

Jika 'jumlahHasil' = 1, 'indexPenduduk' bisa langsung mengambil elemen pertama pada 'hasilCari' karena hanya ada 1 hasil. Dan jika ada 2 atau lebih hasil akan ditampilkan semua nama yang cocok ditambah NIK mereka. Pengguna memilih salah satu kemudian disimpan ke 'indexPenduduk'. Terakhir akun baru dibuat dan dimasukkan ke *array of struct user* dengan *role* bawaan adalah *user*.

```
cout << "\nPilih menu field: \n";
                        cout << "1. Data Penduduk\n";
                        cout << "2. Data akun\n";
                        cout << "3. Data surat\n";
                        cout << "4. Log-out\n";
```

Source Code 3.11 Menu Admin

Admin memiliki kuasa dalam mengatur *database*, dari penduduk, akun, dan surat.

```
case 1 : {
                                while (true) {
                                cout << "\nPilih menu penduduk: \n";
```

```
                                cout << "1. Lihat daftar penduduk\n";
                                cout << "2. Tambah penduduk\n";
                                cout << "3. Edit data penduduk\n";
                                cout << "4. Hapus penduduk\n";
                                cout << "5. Kembali\n";
```

Source Code 3.12 Menu Penduduk

Ini merupakan CRUD utama untuk admin.

```
case 1 : {
                for (int i = 0; i < jumlahPenduduk; i++) {
                     cout << i + 1 << ". NIK: " << dataPenduduk[i].nik <<
                                        ", Nama: " << dataPenduduk[i].nama <<
                                   ", Alamat: " << dataPenduduk[i].alamat
<<
                    ", Nomor Telepon: " << dataPenduduk[i].noTelp << "\n";
                                        }
                                        break;
```

Source Code 3.13 Menampilkan Semua Data Penduduk

Pada pilihan pertama admin bisa melihat daftar penduduk dengan melakukan perulangan sederhana dan setiap *field* penduduk ditampilkan sesuai urutan.

```
string namaBaru, alamatBaru, noTelpBaru;
                                        cout << "Masukkan nama: ";
                                        cin.ignore();
                                        getline(cin, namaBaru);
                                        if (namaBaru == "") {
                                            cout << "Tidak boleh kosong!";
                                            break;
                                        }
                                        cout << "Masukkan alamat: ";
                                        getline(cin, alamatBaru);
                                        if (alamatBaru == "") {
                                            cout << "Tidak boleh kosong!";
                                            break;
                                        }
                                        cout << "Masukkan nomor telepon: ";
                                        getline(cin, noTelpBaru);
                                        if (noTelpBaru == "") {
                                            cout << "Tidak boleh kosong!";
                                            break;
```

Source Code 3.14 Membuat Data Penduduk Baru

Layaknya membuat akun baru, admin bisa menambah data penduduk dengan memasukkan nama, alamat, dan nomor telepon. Masing-masing *field* tidak boleh kosong.

Adapun nomor telepon hanya boleh berisi angka dan tidak boleh sama dengan nomor telepon yang sudah ada.

```
string nikBaru = "PDK" + to_string(++totalPenduduk);
dataPenduduk[jumlahPenduduk] = {nikBaru, namaBaru, alamatBaru, noTelpBaru};
                                        jumlahPenduduk++;
                                        cout << "Berhasil ditambahkan!";
                                        break;
```

Source Code 3.15 Tambahkan Penduduk

Jika semua keperluan terisi, tinggal menambahkan penduduk tersebut ke *database*. NIK merupakan *field* unik yang otomatis terisi ketika membuat data penduduk. Disini 'totalPenduduk' digunakan. Variabel ini berisi kuantitas penduduk saat ini dan yang sudah dihapus. *to_string* akan mengubah integer 'totalPenduduk' menjadi string dan digabung dengan "PDK".

```
string cariNIK;
                cout << "Masukkan NIK dari data penduduk yang ingin diubah: ";
                                        getline(cin, cariNIK);
                                        int indexCari = -1;
                                    for (int i = 0; i < jumlahPenduduk; i++) {
                                        if (dataPenduduk[i].nik == cariNIK) {
                                                indexCari = i;
                                                break;
```

Source Code 3.16 Mencari Penduduk menggunakan NIK

Proses ini akan digunakan saat ingin mengedit data penduduk dan menghapus penduduk. Pertama cari NIK yang diinginkan, selama belum ketemu variabel 'indexCari' akan ditetapkan ke -1 yang menyatakan NIK tidak ditemukan. Pencarian dilakukan di perulangan pada *database* penduduk. Data penduduk yang sesuai disimpan di variabel 'indexCari'

```
cout << "NIK: " << dataPenduduk[indexCari].nik <<
                                        ", " << "Nama: " <<
dataPenduduk[indexCari].nama << ", " <<
                                        "Alamat: " <<
dataPenduduk[indexCari].alamat << ", " <<
                                        "Nomor Telepon: " <<
dataPenduduk[indexCari].noTelp << "\n";
                                        cout << "\nPilih perubahan: \n";
                                        cout << "1. Ubah nama penduduk\n";
```

```
                                        cout << "2. Ubah alamat penduduk\n";
                                    cout << "3. Ubah no telepon penduduk\n";
                                        cout << "4. Kembali\n";
```

Source Code 3.17 Mengedit Data Penduduk

Setelah data penduduk ditemukan, data tersebut akan ditampilkan agar admin bisa mengedit data dengan mudah. Admin dapat mengubah semua *field* kecuali NIK. Proses sama persis seperti pada saat menambah penduduk. Perbedaan hanya pada pembaruan data dengan menimpa data lama dengan data baru. Perubahan juga terjadi pada *database user* mengingat terdapat *nested struct* 'dataDiri'.

```
case 1 : {
                                                string namaGanti;
                                                cout << "Nama baru: ";
                                                cin.ignore();
                                                getline(cin, namaGanti);
                                                if (namaGanti == "") {
                                            cout << "Tidak boleh kosong!";
                                                    break;
                                                }
                                dataPenduduk[indexCari].nama = namaGanti;
                                  for (int i = 0; i < jumlahUser; i++) {
          if (dataUser[i].dataDiri.nik == dataPenduduk[indexCari].nik) {
                                    dataUser[i].dataDiri.nama = namaGanti;
                                                        break;
                                                    }
                                                }
                                                cout << "Berhasil diubah!";
                                                break;
```

Source Code 3.18 Edit Nama

Nama baru akan ditampung pada variabel 'namaGanti', kemudian menimpa data lama pada *database* penduduk. Tidak lupa juga memperbarui data penduduk pada *database user* 'dataDiri' (dataUser[i].dataDiri.nama = namaGanti). Proses sama persis pada mengganti alamat dan nomor telepon.

```
bool adaAdmin = false;
            for (int i = 0; i < jumlahUser; i++) {
              if (dataUser[i].dataDiri.nik == dataPenduduk[indexCari].nik
&&
                                            dataUser[i].role == "admin") {
                                                adaAdmin = true;
```

```
                                                    break;
                                                }
                                            }
                                            if (adaAdmin) {
                        cout << "Tidak bisa hapus penduduk dengan akun admin!\n";
                                                break;
```

Source Code 3.19 Syarat Menghapus Data Penduduk

Admin pun memiliki batasa. Program melarang admin menghapus data penduduk jika penduduk tersebut memiliki akun admin, termasuk mencegah menghapus data penduduk sendiri.

```
if (konfirmasi != 'y') {
                                                cout << "Batal dihapus!";
                                                break;
                                            }
                        string nikHapus = dataPenduduk[indexCari].nik;
                           for (int i = indexCari; i < jumlahPenduduk - 1; i++)
{
                                        dataPenduduk[i] = dataPenduduk[i+1];
                                            }
                                            jumlahPenduduk--;
```

Source Code 3.20 Menghapus Data Penduduk

Admin harus mengkonfirmasi penghapusan untuk mencegah salah hapus. Proses penghapusan dilakukan dengan cara menggeser elemen *array*. Penduduk yang dihapus disimpan pada variabel ‘nikHapus’ diambil dari NIK yang dicari (proses pada *Source Code 3.16*).

Pergeseran dilakukan menggunakan perulangan *for* lagi, indeks dimulai dari indeks dari ‘indexCari’ kenapa? Kita tidak perlu memulai dari indeks 0 karena indeks yang tepat sudah ditemukan. Jika diperhatikan kondisi perulangan ini adalah ‘jumlahPenduduk - 1’, ini bertujuan untuk mencegah *out of bonds*. Bayangkan pergeseran elemen yaitu elemen kanan mengisi elemen kiri, berarti elemen terakhir tidak ada pengisi jika tidak ada -1. Terakhir ‘jumlahPenduduk’ berkurang.

```
for (int i = 0; i < jumlahUser; i++) {
                            if (dataUser[i].dataDiri.nik == nikHapus) {
                              for (int j = i; j < jumlahUser - 1; j++) {
                                        dataUser[j] = dataUser[j+1];
                                                    }
                                                    jumlahUser--;
                                                    break;
```

Source Code 3.21 Penghapusan Berantai

Ketika data penduduk dihapus, maka akun dan semua surat yang penduduk tersebut punya juga akan dihapus. Ada dua perulangan *for* di atas, yang pertama untuk mencari akun dengan membandingkan NIK pada data *nested* di *database user* dengan 'nikHapus'. Perulangan kedua menggunakan 'j' dimulai dari *user* ke-'i' untuk melakukan pergeseran.

```
for (int i = 0; i < jumlahSurat; i++) {
                        if (dataSurat[i].nikPemohon == nikHapus) {
                           for (int j = i; j < jumlahSurat - 1; j++) {
                                    dataSurat[j] = dataSurat[j+1];
                                                }
                                                jumlahSurat--;
                                                i--;
```

Source Code 3.22 Menghapus Semua Surat Sekaligus

Ada sedikit perbedaan pada penghapusan surat. Karena ini menghapus lebih dari satu surat dalam satu waktu, maka terdapat 'i- -'. Visualisasinya:

array : [SRT1(NIK1), SRT2(NIK2), SRT3(NIK1), SRT4(NIK1)]

indeks: 0 1 2 3

Bisa dilihat "NIK1" memiliki 3 surat, dalam kasus ini "NIK1" akan dihapus. Mulai pada indeks ke-0 pada visualisasi. Visualisasi sekarang:

array : [SRT2(NIK2), SRT3(NIK1), SRT4(NIK1)]

indeks: 0 1 2

Maka 'jumlahSurat' berkurang 1, lalu 'i- -' jadi sekarang kita berada di 'i = -1', barulah 'i++' dan kembali ke 'i = 0'. Ini bertujuan untuk mencegah tidak ada surat dari NIK sama yang dilewatkan.

```
case 2 : {
                                while (true) {
                                    cout << "\nPilih menu akun: \n";
                                    cout << "1. Lihat daftar akun\n";
                                    cout << "2. Hapus akun\n";
                                    cout << "3. Kembali\n";
```

Source Code 3.23 Menu Akun

Menu admin berikutnya yaitu akun.

```
case 1 : {
  for (int i = 0; i < jumlahUser; i++) {
    cout << i + 1 << ". " << "Username: " << dataUser[i].usr << ", " <<
   "Role: " << dataUser[i].role << ", " << "NIK: " <<
dataUser[i].dataDiri.nik << ", " << "Nama: " << dataUser[i].dataDiri.nama <<
"\n";
                                                }
                                                break;
```

Source Code 3.24 Menampilkan Daftar Akun

Pada pilihan menampilkan daftar akun, *password* tidak ditampilkan, namun nama dan NIK pemilik akun juga ditampilkan.

```
cout << "Username: " << dataUser[indexCari].usr
     << ", Role: " << dataUser[indexCari].role
     << ", NIK: " << dataUser[indexCari].dataDiri.nik
     << ", Nama: " << dataUser[indexCari].dataDiri.nama;
                                        cout << "Konfirmasi hapus? (y/n): ";
                                                char konfirmasi;
                                                cin >> konfirmasi;
                                                if (konfirmasi != 'y') {
                                                    cout << "Batal dihapus!\n";
                                                    break;
                                                }
                                for (int i = indexCari; i < jumlahUser - 1; i++) {
                                                 dataUser[i] = dataUser[i + 1];
                                                }
                                                jumlahUser--;
                                                if (indexCari < indexUser) {
                                                    indexUser--;
```

Source Code 3.25 Menghapus Akun

Blok kode di atas adalah menghapus akun. Akun dicari berdasarkan *username*, proses sama pada *Source Code 3.16*, hanya berbeda di apa yang dicari. Penghapusan juga tidak bisa dilakukan jika *rolenya* admin. Setelah konfirmasi baru akun dihapus. Metodenya juga menggunakan pergeseran.

Kenapa ada kondisi *if* di akhir? Karena elemen di sebelah kiri 'indexUser' dihapus, semua elemen bergeser 1 ke kiri, termasuk user yang sedang login. Jadi 'indexUser' harus ikut dikurangi 1.

```
string daftarJenis[] {
                                    "Surat Keterangan Domisili",
                                    "Surat Keterangan Tidak Mampu",
                                    "Surat Pengantar KTP",
                                    "Surat Keterangan Kelahiran"
                                };
                                string daftarKeperluan[] {
                                    "Melamar pekerjaan",
                                    "Keperluan pendidikan",
                                    "Keperluan pernikahan",
                                    "Lainnya"
                                };
                                string daftarStatus[] {
                                    "menunggu",
                                    "diproses",
                                    "selesai",
                                    "ditolak"
                                };
```

Source Code 3.26 Array pada Surat

Sebelum ke menu surat admin, di atas merupakan *array* dari *field* surat. Program sudah menetapkan isi dari jenis surat, keperluan, dan status.

```
cout << "Pilih menu surat: \n";
                                    cout << "1. Lihat daftar surat\n";
                                    cout << "2. Update surat\n";
                                    cout << "3. Hapus surat\n";
                                    cout << "4. Kembali \n";
```

Source Code 3.27 Pilihan Menu Surat

Menu surat admin.

```
case 1 : {
                                            if (jumlahSurat == 0) {
                                                cout << "Belum ada
surat!\n";
                                                break;
                                            }
                    for (int i = 0; i < jumlahSurat; i++) {
                       cout << i + 1 << ". ID surat: " << dataSurat[i].idSurat
                       << ", NIK Pemohon: " << dataSurat[i].nikPemohon
                        << ", Jenis Surat: " << dataSurat[i].jenisSurat
                        << ", Keperluan: " << dataSurat[i].keperluan
                        << ", Status: " << dataSurat[i].status
                         << ", Keterangan: " << dataSurat[i].keterangan <<
"\n";
```

```
                                        }
                                        break;
```

Source Code 3.28 Lihat Daftar Surat

Sebelum menampilkan, ‘jumlahSurat’ akan dicek, jika tidak ada surat pada *array* maka surat tidak ditampilkan.

```
cout << "\nPilih perubahan: \n";
                                        cout << "1. Ubah status\n";
                                        cout << "2. Ubah keterangan\n";
                                        cout << "3. Kembali\n";
```

Source Code 3.29 Menu Perubahan Surat Admin

Kita fokus ke menu perubahan. Proses juga sama pada menu-menu sebelumnya dengan mencari ID surat (proses sama pada *Source Code 3.16*). Disini admin hanya bisa mengubah status dan keterangan.

```
case 1: {
                                    int pilihStatus;
                              cout << "Pilih status: \n";
                            for (int i = 0; i < 4; i++) {
                        cout << i + 1 << ". " << daftarStatus[i] << "\n";
                                                }
                                                cout << "Pilih: ";
                                                cin >> pilihStatus;
                                  if (pilihStatus < 1 || pilihStatus > 4) {
                                        cout << "Pilihan tidak valid!\n";
                                                    break;
                                                }
               dataSurat[indexCari].status = daftarStatus[pilihStatus-1];
                                  cout << "Status berhasil diubah!\n";
                                                break;
```

Source Code 3.30 Mengubah Status

Perulangan dilakukan di ‘daftarStatus’ dengan mengakses setiap *array* dari 0-3. Jika pilihan di luar opsi maka kembali ke menu surat. Setelah itu timpa status lama dengan yang baru, menggunakan ‘pilihStatus-1’. Kenapa -1? Tampilan daftar status dimulai dari 1, sementara indeks dimulai dari 0.

```
string isiKtrgn;
                                 cout << "Isi keterangan surat: ";
                                                getline(cin, isiKtrgn);
                                                if (isiKtrgn == "") {
```

```
dataSurat[indexCari].keterangan = "-";
                                    cout << "Keterangan dikosongkan!\n";
                                                        break;
                                                    }
                                        if (isiKtrgn.length() > 50) {
                        cout << "Tidak boleh lebih dari 50 karakter!\n";
                                                        break;
                                                    }
                        dataSurat[indexCari].keterangan = isiKtrgn;
                                cout << "Keterangan berhasil diubah!\n";
                                                    break;
```

Source Code 3.31 Mengubah Keterangan

Keterangan pada surat punya dua ketentuan. Ketika surat dibuat isi keterangan kosong "-" dan tidak boleh lebih dari 50 karakter. Isi dari keterangan menunjukkan progres surat secara lebih detail dari status. Pengeditan seperti biasa dilakukan dengan menimpa.

```
cout << "Konfirmasi hapus? (y/n): ";
                                            char konfirmasi;
                                            cin >> konfirmasi;
                                            if (konfirmasi != 'y') {
                                                cout << "Batal dihapus!\n";
                                                break;
                                            }
                            for (int i = indexCari; i < jumlahSurat - 1; i++)
{
                                              dataSurat[i] = dataSurat[i+1];
                                            }
                                            jumlahSurat--;
                                            cout << "Berhasil dihapus!\n";
                                            break;
```

Source Code 3.32 Hapus Surat secara Individual

Penghapusan surat disini berbeda dengan yang sebelumnya. Di proses ini hanya satu surat yang dihapus. Alurnya; cari ID surat, simpan di 'indexCari', konfirmasi hapus, geser surat yang dihapus dengann yang di sebelahnya, tidak lupa kurangi 'jumlahSurat'.

```
cout << "\nPilih menu: \n";
                        cout << "1. Data sendiri\n";
                        cout << "2. Ganti password\n";
                        cout << "3. Surat menyurat\n";
                        cout << "4. Log-out\n";
```

Source Code 3.33 Menu User

Masuk ke *role* kedua yaitu *user*.

```
case 1: {
     cout << "Data diri kamu\n";
     cout << "NIK: " << dataUser[indexUser].dataDiri.nik << "\n";
     cout << "Nama: " << dataUser[indexUser].dataDiri.nama << "\n";
     cout << "Alamat: " << dataUser[indexUser].dataDiri.alamat << "\n";
     cout << "Nomor Telepon: " << dataUser[indexUser].dataDiri.noTelp <<
"\n";
                                break;
```

Source Code 3.34 Lihat Data Diri Sendiri

Alasan menu ini ada karena ketika pengguna baru membuat akun, dia bisa melihat NIK mereka.

```
string passwordTempo, gantiPassword, konfirmasi;
          cout << "Masukkan password sekarang untuk mengganti password: \n";
                                cin.ignore();
                                getline(cin, passwordTempo);
```

Source Code 3.35 Verifikasi Diri

User bisa mengubah *password* mereka. Tapi harus melakukan verifikasi dengan memasukkan *password* saat ini yang ditampung di 'passwordTempo'. Sama halnya ketika membuat akun, *password* tidak boleh kosong dan minimal berjumlah 3 karakter.

```
if (dataUser[indexUser].pw == passwordTempo) {
                                    cout << "Masukkan password baru: ";
                                    getline(cin, gantiPassword);
                                    if (gantiPassword.length() < 3) {
                          cout << "Panjang password minimal 3 karakter!\n";
                                         break;
                                    }
                                    cout << "Konfirmasi password baru: ";
                                    getline(cin, konfirmasi);
                                    if (gantiPassword == konfirmasi) {
                          dataUser[indexUser].pw = gantiPassword;
                               cout << "Password berhasil diubah!\n";
                                         break;
```

Source Code 3.36 Mengubah Password

Jika verifikasi berhasil *user* bisa memasukkan *password* baru, juga diminta konfirmasi untuk keamanan ekstra, kemudian *password* lama ditimpa.

```
cout << "Pilih menu surat: \n";
                                cout << "1. Liat surat sendiri\n";
                                cout << "2. Buat surat\n";
                                cout << "3. Edit surat\n";
                                cout << "4. Hapus surat\n";
                                cout << "5. Kembali\n";
```

Source Code 3.37 Menu Surat Menyurat

CRUD utama pada *user* adalah surat. *User* bisa melihat, membuat, mengedit, dan menghapus surat.

```
bool adaSurat = false;
for (int i = 0; i < jumlahSurat; i++) {
if (dataSurat[i].nikPemohon == dataUser[indexUser].dataDiri.nik) {
     if (!adaSurat) {
              cout << "Daftar surat: \n";
               adaSurat = true;
                                                }
                 cout << i + 1 << ". ID: " << dataSurat[i].idSurat
                    << ", Jenis: " << dataSurat[i].jenisSurat
                    << ", Keperluan: " << dataSurat[i].keperluan
                    << ", Status: " << dataSurat[i].status
                    << ", Keterangan: " << dataSurat[i].keterangan << "\n";
                                          }
                                 }
                                 if (!adaSurat) {
                              cout << "Kamu belum punya surat!";
                                 }
                                 break;
```

Source Code 3.38 Melihat Surat

Menggunakan perulangan *for* biasa untuk menampilkan surat. Perbandingan menggunakan 'nikPemohon' dengan NIK dari *nested struct* di *database user*. Disini ada dua '!adaSurat', yang pertama berfungsi untuk mencegah *header* "Daftar Surat: " dicetak berkali-kali, jadi ketika sudah dicetak sekali 'adaSurat' menjadi *true*, sisanya melakukan perulangan menampilkan surat. '!adaSurat' kedua untuk menampilkan pesan bahwa *user* saat ini belum ada surat yang diajukan.

```
cout << "Buat surat: ";
                                 int pilihJenis, pilihKeperluan;
                                 string jenisBaru, keperluanBaru;
```

```
                                        cout << "Pilih jenis: \n";
                                        for (int i = 0; i < 4; i++) {
                                cout << i + 1 << ". " << daftarJenis[i] << "\n";
                                        }
                                        cout << "Pilih: ";
                                        cin >> pilihJenis;
                                       if (pilihJenis < 1 || pilihJenis > 4)
{
                                         cout << "Pilihan tidak valid!\n";
                                            break;
                                        }
                                  jenisBaru = daftarJenis[pilihJenis-1];
```

Source Code 3.39 Membuat Surat

Proses membuat surat mirip dengan memperbarui status surat pada menu edit surat admin. *User* hanya bisa memilih pada daftar. Di atas adalah pemilihan jenis surat yang disimpan di variabel 'jenisBaru' sesuai 'pilihJenis'.

```
cout << "Pilih: ";
                                        cin >> pilihKeperluan;
                                if (pilihKeperluan < 1 || pilihKeperluan > 4) {
                                       cout << "Pilihan tidak valid!\n";
                                            break;
                                        }
```

Source Code 3.40 Buat Keperluan

Lakukan proses yang sama pada menambah keperluan, pilih satu di antara semua daftar.

```
string nikPemohonSementara = dataUser[indexUser].dataDiri.nik;
                                        string idBaru = "SRT" +
to_string(++totalSurat);
                                       dataSurat[jumlahSurat] = {idBaru,
nikPemohonSementara, jenisBaru, keperluanBaru, "menunggu", "-"};
                                        jumlahSurat++;
```

Source Code 3.41 Tambah Surat ke Database

Untuk menyimpan 'nikPemohon', program membaca dari *database user* yang sedang *login* dan mengambil NIK-nya. Pembuatan ID surat juga sama seperti NIK, ambil 'totalSurat' dan jadikan *string*. Status akan ditetapkan ke 'menunggu'.

```
if (dataSurat[indexCari].status != "menunggu") {
                                cout << "Surat tidak bisa diedit!";
                                            break;
                                        }
                                    cout << "Pilih perubahan: \n";
                                    cout << "1. Jenis surat\n";
                                    cout << "2. Keperluan surat\n";
                                    cout << "3. Kembali\n";
```

Source Code 3.42 Perubahan Surat

Cari ID seperti pada proses-proses sebelumnya, simpan di 'indexCari'. *User* dapat mengubah jenis atau keperluan surat dengan **catatan** status surat tersebut masih 'menunggu'. Selebihnya surat tidak dapat diedit.

```
dataSurat[indexCari].jenisSurat = daftarJenis[gantiJenis-1];
                                cout << "Jenis surat berhasil diubah!\n";
```

Source Code 3.43 Mengedit Jenis Surat

```
dataSurat[indexCari].keperluan = daftarKeperluan[gantiKeperluan-1];
                            cout << "Keperluan surat berhasil diubah!\n";
```

Source Code 3.44 Mengedit Keperluan Surat

Timpa jenis/keperluan lama dengan yang baru.

```
if (dataSurat[indexCari].status != "menunggu") {
                                cout << "Surat tidak bisa dihapus!\n";
                                            break;
                                        }
                                    cout << "Yakin ingin hapus surat? (y/n): ";
                                        char konfirmasi;
                                        cin >> konfirmasi;
                                        if (konfirmasi != 'y') {
                                            cout << "Batal dihapus!";
                                            break;
                                        }
                            for (int i = indexCari; i < jumlahSurat - 1; i++) {
                                            dataSurat[i] = dataSurat[i+1];
                                        }
                                        jumlahSurat--;
                                        cout << "Surat berhasil dihapus!\n";
                                        break;
```

Source Code 3.45 Hapus Surat Versi User

Proses hapus surat pada *user* sama persis dengan hapus surat pada admin. Perbedaan tepat di status surat yang jika selain 'menunggu' maka surat tidak bisa dihapus.

4. Hasil Output

```
Selamat datang di Website Balai Kota!

Silahkan login untuk melanjutkan
1. Login
2. Register
3. Keluar
Pilih:
```

Gambar 4.1 Tampilan Auth

```
Pilih: 2
Masukkan username baru: am
Masukkan password baru: 013
Masukkan nama lengkap: amat
Ditemukan 2 penduduk dengan nama sama:
1. amat - PDK3
2. amat - PDK4
Pilih nomor: 1
Registrasi berhasil!
Nik kamu: PDK3
```

Gambar 4.2 Registrasi dengan Nama Sama

```
Login berhasil! Selamat datang, r!


Pilih menu field:
1. Data Penduduk
2. Data akun
3. Data surat
4. Log-out
```

Gambar 4.3 Menu Admin

```
Pilih menu penduduk:
1. Lihat daftar penduduk
2. Tambah penduduk
3. Edit data penduduk
4. Hapus penduduk
5. Kembali
```

Gambar 4.4 Menu Penduduk

```
Pilih: 1
1. NIK: PDK1, Nama: radja, Alamat: gg bersatu, Nomor Telepon: 08111
2. NIK: PDK2, Nama: amat, Alamat: gg bersama, Nomor Telepon: 08112
3. NIK: PDK3, Nama: amat, Alamat: gg batu, Nomor Telepon: 08113
4. NIK: PDK4, Nama: amat, Alamat: gg putih, Nomor Telepon: 08114
```

Gambar 4.5 Daftar Penduduk

```
Pilih: 2
Masukkan nama: Omar
Masukkan alamat: gg bersatu
Masukkan nomor telepon: 08115
Berhasil ditambahkan!
```

Gambar 4.6 Tambah Penduduk

```
Pilih: 3
Masukkan NIK dari data penduduk yang ingin diubah: PDK5
NIK: PDK5, Nama: Omar, Alamat: gg bersatu, Nomor Telepon: 08115

Pilih perubahan:
1. Ubah nama penduduk
2. Ubah alamat penduduk
3. Ubah no telepon penduduk
4. Kembali
Pilih: 2
Alamat baru: gg matahari
Berhasil diubah!
```

Gambar 4.7 Mengedit Data Penduduk

```
Pilih: 4
Masukkan NIK dari data penduduk yang ingin dihapus: PDK4
NIK: PDK4, Nama: amat, Alamat: gg putih, Nomor Telepon: 08114
Konfirmasi hapus? (y/n): y
Berhasil dihapus!
```

Gambar 4.8 Menghapus Penduduk

```
Pilih: 1
1. NIK: PDK1, Nama: radja, Alamat: gg bersatu, Nomor Telepon: 08111
2. NIK: PDK2, Nama: amat, Alamat: gg bersama, Nomor Telepon: 08112
3. NIK: PDK3, Nama: amat, Alamat: gg batu, Nomor Telepon: 08113
4. NIK: PDK5, Nama: Omar, Alamat: gg matahari, Nomor Telepon: 08115
```

Gambar 4.9 Daftar Penduduk Setelah Edit dan Hapus

```
Pilih menu akun:
1. Lihat daftar akun
2. Hapus akun
3. Kembali
```

Gambar 4.10 Tampilan Menu Akun

```
Pilih: 1
1. Username: r, Role: admin, NIK: PDK1, Nama: radja
2. Username: a, Role: user, NIK: PDK2, Nama: amat
3. Username: am, Role: user, NIK: PDK3, Nama: amat
```

Gambar 4.11 Daftar Akun

```
Pilih menu surat:
1. Lihat daftar surat
2. Update surat
3. Hapus surat
4. Kembali
Pilih: 1
1. ID surat: SRT1, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan:
Keperluan Pendidikan, Status: diproses, Keterangan: Dalam pengecekkan
2. ID surat: SRT2, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan:
Keperluan usaha, Status: menunggu, Keterangan: -
```

Gambar 4.12 Menu Surat dan Lihat Daftar Surat

```
Pilih: 2
Masukkan ID dari surat yang ingin diupdate: SRT1
ID surat: SRT1, NIK Pemohon: PDK2, Jenis Surat: Surat Keterangan Tidak Mampu, Keperluan: Kep
erluan Pendidikan, Status: selesai, Keterangan: Dalam pengecekkan

Pilih perubahan:
1. Ubah status
2. Ubah keterangan
3. Kembali
Pilih: 2
Isi keterangan surat: Sudah dikirimkan
Keterangan berhasil diubah!
Pilih menu surat:
1. Lihat daftar surat
2. Update surat
3. Hapus surat
4. Kembali
Pilih: 3
Masukkan ID dari surat yang ingin dihapus: SRT2
ID surat: SRT2, Jenis: Surat Keterangan Tidak Mampu, Status: menunggu
Konfirmasi hapus? (y/n): y
Berhasil dihapus!
```

Gambar 4.13 Edit dan Hapus Surat

```
Silahkan login untuk melanjutkan
Username: am
Password: 013

Login berhasil! Selamat datang, am!


Pilih menu:
1. Data sendiri
2. Ganti password
3. Surat menyurat
4. Log-out
```

Gambar 4.14 Menu User

```
Data diri kamu
NIK: PDK3
Nama: amat
Alamat: gg batu
Nomor Telepon: 08113
```

Gambar 4.15 Data Sendiri

```
Pilih: 2
Masukkan password sekarang untuk mengganti password:
013
Masukkan password baru: 015
Konfirmasi password baru: 015
Password berhasil diubah!
```

Gambar 4.16 Mengubah Password

```
Pilih: 2
Buat surat: Pilih jenis:
1. Surat Keterangan Domisili
2. Surat Keterangan Tidak Mampu
3. Surat Pengantar KTP
4. Surat Keterangan Kelahiran
Pilih: 3
Pilih keperluan:
1. Melamar pekerjaan
2. Keperluan pendidikan
3. Keperluan pernikahan
4. Lainnya
Pilih: 4
Surat berhasil ditambahkan!
```

Gambar 4.17 Buat Surat

```
Masukkan ID surat yang ingin diubah:
SRT4
Pilih perubahan:
1. Jenis surat
2. Keperluan surat
3. Kembali
Pilih: 2
Pilih keperluan:
1. Melamar pekerjaan
2. Keperluan pendidikan
3. Keperluan pernikahan
4. Lainnya
Pilih: 2
Keperluan surat berhasil diubah!
```

Gambar 4.18 Edit Surat User

```
Daftar surat:
2. ID: SRT4, Jenis: Surat Pengantar KTP, Keperluan: Keperluan pendidikan, Status: menunggu,
Keterangan: -
```

Gambar 4.19 Daftar Surat User

5. Langkah-langkah GIT

Sebelum melakukan proses *Git*, kita harus memiliki akun GitHub dan membuat *repository* baru. Lalu siapkan *folder* yang akan di git dan buka Command Prompt, PowerShell, atau semacamnya untuk melakukan *git*.

5.1 Git Init

```
PS D:\praktikum-apl> git init
Initialized empty Git repository in D:/praktikum-apl/.git/
```

Gambar 5.1.1 Proses Git Init

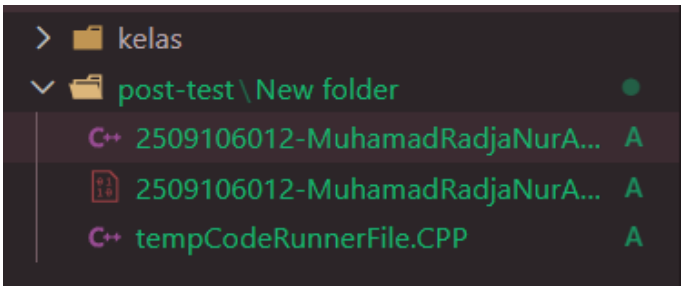
Git init bisa dilakukan terminal pada VSCode. Cukup ketik "git init" pada terminal untuk menginisialisasi/membuat *repository git* di *folder* lokal dan memantau setiap perubahan yang terjadi di dalam *folder*.

5.2 Git Add

```
PS D:\praktikum-apl> git add .
```

Gambar 5.2.1 Proses Git Add

Git add berfungsi untuk memindah semua perubahan di area kerja ke indeks, ada dua cara untuk melakukan *git add*, yang pertama dengan mengetikkan nama *file* yang ingin di add. Jika kalian ingin menambahkan banyak *file* sekaligus, ketik "." Setelah *add* maka semua *file* yang di dalam area kerja akan ditambahkan.



Gambar 5.2.2 Setelah Git Add

5.3 Git Commit

```
PS D:\praktikum-apl> git commit -m "finish pt 1"
[master (root-commit) e8f9088] finish pt 1
 3 files changed, 98 insertions(+)
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.CPP
 create mode 100644 post-test/New folder/2509106012-MuhamadRadjaNurAkbar-PT-1.exe
 create mode 100644 post-test/New folder/tempCodeRunnerFile.CPP
```

Gambar 5.3.1 Proses Commit

Git commit berfungsi untuk mengkonfirmasi setiap perubahan pada *repository* kalian dengan mengetikkan "git commit -m 'pesan yang ingin ditulis' " .

5.4 Git Remote

```
PS D:\praktikum-apl> git remote add origin https://github.com/MRadjaNurAkbarA25/
praktikum-apl.git
```

Gambar 5.4.1 Git Remote

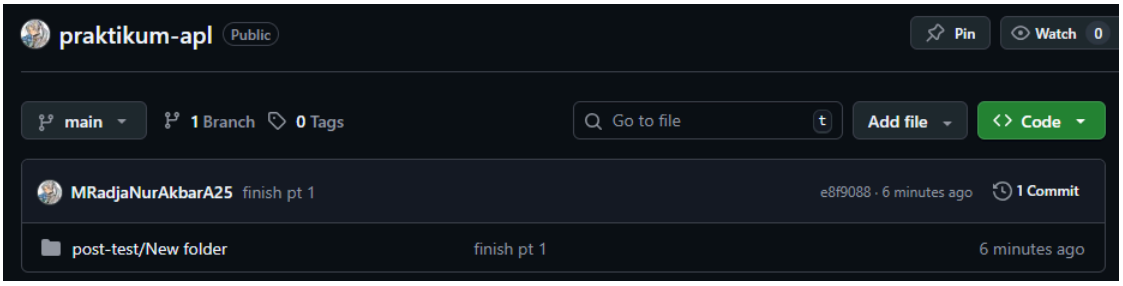
Git remote berfungsi untuk menghubungkan *repository* lokal ke *repository* yang ada di GitHub kalian, caranya dengan mengetik "git remote add origin 'link *repository* GitHub kalian' " .

5.5 Git Push

```
PS D:\praktikum-apl> git push -u origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 667.93 KiB | 7.95 MiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
```

Gambar 5.5.1 Git Push

Git push berfungsi untuk mengunggah *file* lokal tadi ke GitHub kalian dengan mengetikkan "git push -u origin main. Kembali ke *repository* di GitHub kalian, klik *refresh* dan *file* sudah terunggah.



Gambar 5.5.2 File Terunggah